SQL Script

רשת המרכולים "שפע יששכר"

- **INDEX**

Part 1 – Data Overview

a) geolocation table: Data Quality & Entity Profiling

1. Checking NULL VALUES
2. Checking distinct device_id
3. Checking distinct roles
4. Checking distinct areas
5. Timeline & operational consistency check
6. Outliers in timestamp
7. Checking outliers in dow(day of week) = wednesday (4)
8. Total outliers per role

b) log_sales table: : Data Quality & Entity Profiling

9. Checking ZERO or NULL VALUES in sales
10. Outliers in timestamp
11. Clients per day of week overall
12. Clients per month
13. Clients per month and day of week
14. Daily sales
15. Monthly sales
16. Payment Methods
17. Sum of each payment method
18. Daily Payment Methods
19. Customer Behavior & App Engagement
20. App Penetration in Sales

Part 2 – Key Metrics & KPIs

Dealing with outliers and assessing application performance integrity

21. Identifying and treating outliers in location transmission accuracy level
22. Identify the role for which the m_accuracy is consistently atypical. Try to speculate why.
23. Validation of dwell_minutes (In-Store Consistency Check)
24. App capture rate – In-Store only (Official Metric)
25. Quantifying the impact of parking on dwell time
26. Sales activity coverage analysis
27. Daily frequency of customers
28. Weekday operational coverage validation
29. Anomaly Detection (Zero Activity & Extreme Demand)
30. Days when the store was supposed to be open, but no sales occurred
31. Excessive activity days only
32. Sales closure validation using customer geolocation activity
33. Store operational windows: Cross-Signal Validation
34. Customer Traffic Density & Behavioral Profiling
35. Supplier Behavioral Profiling
36. Detection of Scheduled Deliveries That Did Not Occur
37. Deliveries Outside the Expected Schedule
38. Identify the Top 5 customers with the highest visit frequency.
39. Identify the Top 5 customers with the highest cumulative expenditure throughout the entire period.
40. Identifies customers who are both top visitors and top spenders
41. Ranking customers by total spending with ticket analysis
42. Financial customer segmentation: ticket × frequency

43. Revenue Impact Analysis by Segment
44. Dwell Time and Efficiency Analysis by Segment

Part 3 – Business Recommendations

45. Combined Customers × Transactions: Percentile-Based Composite Activity Index
46. Customers-Only Version (Sanity Check)
47. Pre-Holiday vs Regular Day Segmentation
48. Hourly Checkout Demand
49. Observed Cashier Capacity (Throughput)
50. Dwell Time Validation
51. Recommended number of cashiers per hour
52. Self-checkout investment windows
53. Recommended cashiers vs. observed staffing
54. Hourly overstaffing / understaffing detection (Weekday × Hour)
55. Staffing analysis with peak / dead hour classification (p75 Demand)
56. Self-checkout potential based on low operational utilization
57. Integrated staffing × demand × self-checkout decision KPI

Part 4 – Regression Analysis

58. Data Preparation & Business Logic
59. Data Preparation and Modeling Decisions

Part 1 - Data Overview

I. Exploratory Data Analysis (EDA)

a) geolocation table: Data Quality & Entity Profiling

1. Checking NULL VALUES

```
select
  sum(case when device_id is null then 1 else 0 end) as null_device_id,
  sum(case when lat is null then 1 else 0 end) as null_lat,
  sum(case when lon is null then 1 else 0 end) as null_lon,
  sum(case when timestamp is null then 1 else 0 end) as null_timestamp,
  sum(case when accuracy_m is null then 1 else 0 end) as null_accuracy_m,
  sum(case when role is null then 1 else 0 end) as null_role,
  sum(case when area is null then 1 else 0 end) as null_area,
 count(device_id) as total_id
from `bqproj-435911.Final_Project_2025.geolocation`;
--> NO NULL VALUES
```

2. Checking distinct device_id

```
select distinct device_id
from `bqproj-435911.Final_Project_2025.geolocation`;

select distinct device_id
from `bqproj-435911.Final_Project_2025.geolocation`
where accuracy_m <= 30;
--> TOTAL = 1080
```

3. Checking distinct roles

```
select role,
   count(distinct device_id) as total_devices_per_role
from `bqproj-435911.Final_Project_2025.geolocation`
group by role;
```

Row	role	total_devices_per_role
1	butcher	4
2	not_paying	276
3	one_time_customer	409
4	security_guy	4
5	cashier	15
6	delivery_guy	8
7	manager	1
8	general_worker	10
9	repeat_customer	352
10	senior_general_worker	1

Row	area
1	CASH_REGISTERS
2	BUTCHERY
3	HEAD_OFFICE
4	SUPERMARKET
5	PARKING
6	WAREHOUSE

4. Checking distinct areas

```
select distinct area
from `bqproj-435911.Final_Project_2025.geolocation`;
```

5. Timeline & operational consistency check

```
select
  min(timestamp) as first_date,
  max(timestamp) as last_date
from `bqproj-435911.Final_Project_2025.geolocation`;
--> This validation confirms that the data reflects the expected operational calendar.
```

6. Outliers in timestamp

```
with data_with_flag as (
    select
        timestamp,
        extract(dayofweek from timestamp) as dow,
        time(timestamp) as t,
        role,
        area,
        case
            -- special rule for delivery_guy
            -- monday (2) and thursday (5) deliveries around 06:00
            when role = 'delivery_guy'
                 and extract(dayofweek from timestamp) in (2, 5)
                 and time(timestamp) between time '05:30:00' and time '07:00:00'
            then 0  -- not an outlier

            -- saturday (dow = 7): store closed
            when extract(dayofweek from timestamp) = 7 then 1

            -- sunday (1), monday (2), tuesday (3): 07:30–21:00
            when extract(dayofweek from timestamp) in (1, 2, 3)
                 and (time(timestamp) < time '07:25:00'
                      or time(timestamp) > time '21:30:00')
            then 1

            -- wednesday (4), thursday (5): 07:30–22:00
            when extract(dayofweek from timestamp) in (4, 5)
                 and (time(timestamp) < time '07:25:00'
                      or time(timestamp) > time '22:30:00')
            then 1

            -- friday (6): 07:00–15:00
            when extract(dayofweek from timestamp) = 6
                 and (time(timestamp) < time '06:55:00'
                      or time(timestamp) > time '15:10:00')
            then 1
            else 0
        end as is_outlier_flag
    from `bqproj-435911.Final_Project_2025.geolocation`
)
select
    timestamp,
    extract(dayofweek from timestamp) as dow,
    role,
    area,
    'outlier' as status
from data_with_flag
where is_outlier_flag = 1
order by dow, timestamp;
```

7. Checking outliers in dow(day of week) = wednesday (4)

```
--> RESULT: "THERE'S NO DATA TO DISPLAY"
with data_with_flag as (
    select
        timestamp,
        extract(dayofweek from timestamp) as dow,
        time(timestamp) as t,
        role,
        area,
          case
            -- special rule for delivery_guy
            -- monday (2) and thursday (5) deliveries around 06:00
            when role = 'delivery_guy'
                 and extract(dayofweek from timestamp) in (2, 5)
                 and time(timestamp) between time '05:30:00' and time '07:00:00'
            then 0  -- not an outlier

            -- saturday (dow = 7): store closed
            when extract(dayofweek from timestamp) = 7 then 1

            -- sunday (1), monday (2), tuesday (3): 07:30–21:00
            when extract(dayofweek from timestamp) in (1, 2, 3)
                 and (time(timestamp) < time '07:25:00'
                      or time(timestamp) > time '21:15:00')
            then 1

            -- wednesday (4), thursday (5): 07:30–22:00
            when extract(dayofweek from timestamp) in (4, 5)
                 and (time(timestamp) < time '07:25:00'
                      or time(timestamp) > time '22:30:00')
            then 1

            -- friday (6): 07:00–15:00
            when extract(dayofweek from timestamp) = 6
                 and (time(timestamp) < time '06:55:00'
                      or time(timestamp) > time '15:10:00')
            then 1
            else 0
        end as is_outlier_flag
    from `bqproj-435911.Final_Project_2025.geolocation`
),

outliers as ( select
    timestamp,
    extract(dayofweek from timestamp) as day_of_week,
    role,
    area,
    'outlier' as status
from data_with_flag
where is_outlier_flag = 1
order by timestamp)

select role, area, timestamp, day_of_week
from outliers
where day_of_week = 4;
```

8. Total outliers per role

```
with data_with_flag as (
    select
        timestamp,
```

```
        extract(dayofweek from timestamp) as dow,
        time(timestamp) as t,
        role,
        area,
          case
            -- special rule for delivery_guy
            -- monday (2) and thursday (5) deliveries around 06:00
            when role = 'delivery_guy'
                 and extract(dayofweek from timestamp) in (2, 5)
                 and time(timestamp) between time '05:30:00' and time '07:00:00'
            then 0  -- not an outlier

            -- saturday (dow = 7): store closed
            when extract(dayofweek from timestamp) = 7 then 1

            -- sunday (1), monday (2), tuesday (3): 07:30–21:00
            when extract(dayofweek from timestamp) in (1, 2, 3)
                 and (time(timestamp) < time '07:25:00'
                      or time(timestamp) > time '21:15:00')
            then 1

            -- wednesday (4), thursday (5): 07:30–22:00
            when extract(dayofweek from timestamp) in (4, 5)
                 and (time(timestamp) < time '07:25:00'
                      or time(timestamp) > time '22:30:00')
            then 1

            -- friday (6): 07:00–15:00
            when extract(dayofweek from timestamp) = 6
                 and (time(timestamp) < time '06:55:00'
                      or time(timestamp) > time '15:10:00')
            then 1
            else 0
        end as is_outlier_flag
    from `bqproj-435911.Final_Project_2025.geolocation`
),
outliers as ( select
    timestamp,
    extract(dayofweek from timestamp) as day_of_week,
    role,
    area,
    'outlier' as status
from data_with_flag
where is_outlier_flag = 1
order by timestamp
)
select role, area,
count(role) as total_outliers
from outliers
group by 1,2;
```

Row	role	area	total_outliers
1	cashier	CASH_REGISTERS	56
2	security_guy	PARKING	1629

Conclusion:

--> After running the query with the store's exact operating hours, I noticed that some employees arrive before the store opens (around 5 minutes) and some employees leave the store

up to an hour after closing (cashiers and security_guy). For this reason, I decided not to filter these outliers because I believe they are part of the store's operation.
--> What caught my attention is that there are no outliers on WEDNESDAY (day_of_week = 4)

b) log_sales table: : Data Quality & Entity Profiling

9. Checking ZERO or NULL VALUES in sales

```
select
  sum(case when sale_id is null then 1 else 0 end) as null_sale_id,
  sum(case when customer_id is null then 1 else 0 end) as null_customer_id,
  sum(case when subtotal is null then 1 else 0 end) as null_subtotal,
  sum(case when tax is null then 1 else 0 end) as null_tax,
  sum(case when total is null then 1 else 0 end) as null_total,
  sum(case when subtotal < 0 then 1 else 0 end) as neg_subtotal,
  sum(case when tax < 0 then 1 else 0 end) as neg_tax,
  sum(case when total < 0 then 1 else 0 end) as neg_total,
 count(sale_id) as total_id
from `bqproj-435911.Final_Project_2025.log_sales`;
```

--> Nulls in customer_id represent non-app users (unregistered customers).
--> sales_id has been validated as a unique Primary Key (PK).

10. Outliers in timestamp

```
--> RESULT: NO OUTLIERS
with data_with_flag as (
    select
        timestamp,
        extract(dayofweek from timestamp) as dow,
        time(timestamp) as t,
        case
            -- saturday (dow = 7): store closed
            when extract(dayofweek from timestamp) = 7 then 1

            -- sunday, monday, tuesday: 07:30-21:00
            when extract(dayofweek from timestamp) in (1, 2, 3)
                 and (time(timestamp) < time '07:30:00'
                      or time(timestamp) > time '21:00:00') then 1

            -- wednesday, thursday: 07:30-22:00
            when extract(dayofweek from timestamp) in (4, 5)
                 and (time(timestamp) < time '07:30:00'
                      or time(timestamp) > time '22:00:00') then 1

            -- friday: 07:00-15:00
            when extract(dayofweek from timestamp) = 6
                 and (time(timestamp) < time '07:00:00'
                      or time(timestamp) > time '15:00:00') then 1

            else 0
        end as is_outlier_flag
    from `bqproj-435911.Final_Project_2025.log_sales`
)

select
    timestamp,
    extract(dayofweek from timestamp) as day_of_week,
    'outlier' as status
from data_with_flag
```

```
where is_outlier_flag = 1
order by timestamp;
```

11. Clients per day of week overall

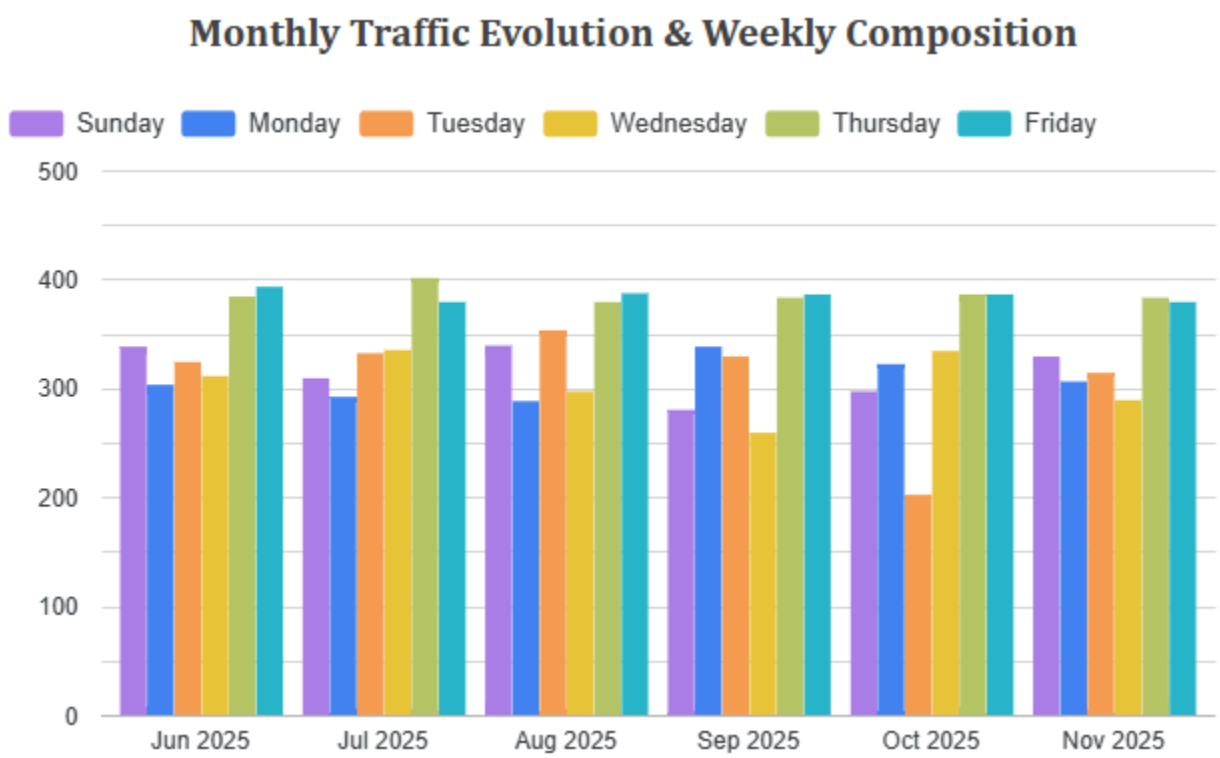
```
select
 extract(dayofweek from timestamp) as sequence_day,
 format_date('%A', date(timestamp)) as week_day,
 case when extract(dayofweek from timestamp) = 6 or extract(dayofweek from timestamp) = 7 then
'weekend' else 'week' end as type_of_day,
 count(distinct customer_id) as daily_customers,
 count(*) as total_transactions
from `bqproj-435911.Final_Project_2025.log_sales`
group by sequence_day,
     week_day,
     type_of_day
order by sequence_day;
```

12. Clients per month

```
select
  format_date('%m', date(timestamp)) as month_clients,
  count(distinct customer_id) as daily_clients
from `bqproj-435911.Final_Project_2025.log_sales`
group by month_clients
order by month_clients;
```

13. Clients per month and day of week

```
select
 format_date('%m', date(timestamp)) as month_clients,
 format_date('%A', date(timestamp)) as week_day,
 count(distinct customer_id) as daily_clients
from `bqproj-435911.Final_Project_2025.log_sales`
group by month_clients, week_day
order by month_clients;
```

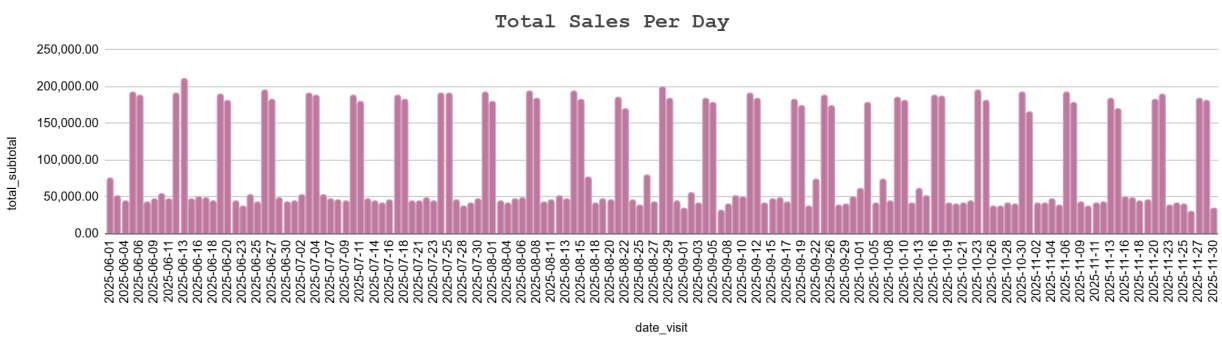


This chart provides a detailed view of store traffic trends from June to November 2025. Here is a brief analysis of the findings:

- Weekly Peak Period: The chart reveals a consistent surge in traffic toward the end of the week, with Thursday (Green) and Friday (Cyan) reaching the highest customer counts across all observed months.
- Stability and Growth: Overall traffic remains relatively stable throughout the period, maintaining a baseline of approximately 300 to 400 customers per day.
- Tuesday Volatility: While most days show steady performance, there is a significant and notable dip in Tuesday (Orange) traffic during October 2025 compared to the preceding and following months.
- Operational Insight: The dominance of end-of-week traffic suggests that staffing and operational support should be heavily prioritized on Thursdays and Fridays to accommodate the consistently higher volume.

14. Daily sales

```
select
  date(timestamp) as date_visit,
  format_date('%A', date(timestamp)) as week_day,
  round (sum(subtotal), 2) as total_subtotal,
  round (sum(tax), 2) as total_tax,
  round (sum(total), 2) as total
from `bqproj-435911.Final_Project_2025.log_sales`
group by date_visit, week_day
order by date_visit;
```



15. Monthly sales

```
select
    format_date('%Y-%m', date(timestamp)) as sales_month, -- year-month identifier
    round(sum(subtotal), 2) as total_subtotal,             -- total net sales before tax
    round(sum(tax), 2) as total_tax,                       -- total tax collected
    round(sum(total), 2) as total_revenue                  -- total gross revenue
from `bqproj-435911.Final_Project_2025.log_sales`
group by sales_month
order by sales_month;
```

16. Payment Methods

```
select payment_method,
  count (payment_method) as total_transactions,
  round(count(payment_method) * 100 / sum(count(payment_method)) over(), 2) as percentage
from `bqproj-435911.Final_Project_2025.log_sales`
group by 1
order by 2 desc;
```

Row	payment_method	total_transactions	percentage
1	credit_card	15162	43.83
2	debit_card	14989	43.33
3	cash	2232	6.45
4	mobile_pay	2206	6.38

17. Sum of each payment method

```
select payment_method,
  round(sum (subtotal),2) as subtotal_payment,
  round(sum (tax),2) as tax_payment,
  round(sum (total),2) as total_payment
from `bqproj-435911.Final_Project_2025.log_sales`
group by 1
order by 2 desc;
```

18. Daily Payment Methods

```
select
  date(timestamp) as date_visit,
  format_date('%A', date(timestamp)) as week_day,
  payment_method,
  round (sum(subtotal), 2) as total_subtotal,
  round (sum(tax), 2) as total_tax,
  round (sum(total), 2) as total
from `bqproj-435911.Final_Project_2025.log_sales`
group by payment_method, date_visit, week_day
order by date_visit, total_subtotal;
```

19. Customer Behavior & App Engagement

```
with role_counts as (
  -- This query counts geolocation activations per customer role
  select
    role,
    count(*) as total_activations
  from `bqproj-435911.Final_Project_2025.geolocation`
  where accuracy_m <= 30 -- Filtering for accuracy yields 893,044 total (68.24% of raw data)
  group by 1
),
percentage_calculation as (
  -- Calculates the overall total and the percentage distribution for each role
  select
    role,
    total_activations,
    sum(total_activations) over() as overall_total,
    (total_activations * 100.0 / sum(total_activations) over ()) as percentage_of_total
  from role_counts
)
select
  role,
  total_activations,
  overall_total,
  round(percentage_of_total, 2) as percentage_of_total
from percentage_calculation
where role in ('repeat_customer', 'one_time_customer', 'not_paying')
order by total_activations DESC;
```

role	total_activations	overall_total	percentage_of_to...
repeat_customer	538944	893044	60.35
not_paying	65740	893044	7.36
one_time_customer	17166	893044	1.92

Conclusion:

Dominant Role: The analysis shows that Repeat Customers represent the vast majority of store engagement, accounting for 60.35% of total geolocation activations.
Engagement Insight: The low percentage of "one-time customers" (1.92%) compared to "repeat customers" suggests a highly loyal customer base that consistently interacts with the store's digital ecosystem.
Data Quality: Applying an accuracy filter of <=30 meters ensures that the analysis focuses on high-fidelity indoor positioning, retaining approximately 68% of the raw geolocation data for reliable store-traffic mapping(detailed calculations for these figures are provided in Part 2).

20. App Penetration in Sales

```
select
    count(*) as total_sales,
    sum(case when customer_id is not null then 1 else 0 end) AS app_customers,
    round((sum(case when customer_id is not null then 1 else 0 end) * 100.0 / count(*)), 2) AS
percentage_app_customers,
    sum(case when customer_id is null then 1 else 0 end) AS no_app_customers,
    round((sum(case when customer_id is null then 1 else 0 end) * 100.0 / count(*)), 2) AS
percentage_no_app_customers
from `bqproj-435911.Final_Project_2025.log_sales`;
```

total_sales	app_customers	percentage_app_customers	no_app_customers	percentage_no_app_customers
34589	30993	89.6	3596	10.4

Conclusion:
The results indicate that the mobile app has very high adoption among paying customers. Approximately 89.6% of all sales are associated with identified app users, while only 10.4% of transactions come from customers without the app. This suggests that the app is strongly embedded in the customer journey and effectively captures the vast majority of purchasing activity, making it a reliable source for behavioral and transactional analysis.

Part 2 - Key Metrics & KPIs

I. Dealing with outliers and assessing application performance integrity

21. Identifying and treating outliers in location transmission accuracy level

When a device transmits its location, it calculates the deviation range (m_accuracy), which represents the level of location precision in meters. The lower the value, the more precise the location, and vice versa.

- Calculate how many transmissions in the data are defined as outliers–that is, those where the m_accuracy value is greater than 30 meters.
- Check if this is a significant amount relative to the total data.
- Explain what, in your opinion, should be done with these atypical transmissions (mitigate, adjust, or exclude) and why. Proceed according to your decision.

```sql
select
    count(*) as total_signal,
    count(case when accuracy_m <= 30 then 1 else null end) as filtered_accuracy_m,
    round(count(case when accuracy_m <= 30 then 1 else null end) * 100.0 / count(*), 2) AS
percentage_filtered_accuracy
from `bqproj-435911.Final_Project_2025.geolocation`;
```

Row	total_signal	filtered_accuracy_m	percentage_filtered_accuracy
1	1308624	893044	68.24

Conclusion:

--> I analyzed the distribution of location accuracy (accuracy_m) to identify outlier transmissions.
--> Transmissions with accuracy_m > 30 meters were classified as outliers due to low spatial precision.
--> Out of 1,308,624 total signals, 68.24% met the accuracy threshold (≤ 30m), while approximately 31.76% were considered low-precision signals.
--> Given the significant proportion of imprecise data and its potential to distort in-store presence and dwell time analyses, I excluded transmissions with accuracy_m > 30 from subsequent analyses.
--> This decision improves spatial reliability while preserving sufficient data volume for robust customer behavior insights.

22. Identify the role for which the m_accuracy is consistently atypical. Try to speculate why.

a) Identify all distinct role and area combinations in the geolocation data

- This step is essential to understand the operational context of each role.
- Different roles are expected to operate in different physical areas(e.g., customers in the supermarket and parking, cashiers at registers, security staff in parking areas).
- Mapping roles to areas allows me to:

  -- Validate that roles appear in logically consistent locations
  -- Avoid misleading conclusions when analyzing GPS accuracy outliers
  -- Ensure that outlier detection is interpreted within the correct physical context
- This step directly supports the next analysis, where I identify which role exhibits the highest proportion of m_accuracy outliers.

```sql
select distinct role, area
from `bqproj-435911.Final_Project_2025.geolocation`
order by 1;
```

Row	role	area
1	butcher	BUTCHERY
2	cashier	CASH_REGISTERS
3	delivery_guy	WAREHOUSE
4	general_worker	SUPERMARKET
5	manager	HEAD_OFFICE
6	manager	SUPERMARKET
7	not_paying	SUPERMARKET
8	not_paying	PARKING
9	one_time_customer	PARKING
10	one_time_customer	SUPERMARKET
11	one_time_customer	CASH_REGISTERS
12	repeat_customer	SUPERMARKET
13	repeat_customer	CASH_REGISTERS
14	repeat_customer	PARKING
15	security_guy	PARKING
16	senior_general_worker	SUPERMARKET

b) Accuracy outlier analysis by role

```
with role_area_data as (
    -- applies filters to ensure we are analyzing the role within its expected context
    select
        role,
        accuracy_m
    from
        `bqproj-435911.Final_Project_2025.geolocation`
    where
        -- filter for customers INSIDE the store, where precision matters
        (role in ('one_time_customer', 'repeat_customer', 'not_paying') and area in
('SUPERMARKET', 'CASH_REGISTERS', 'PARKING'))
        -- filter for employees in their work areas
        or (role = 'butcher' and area = 'BUTCHERY')
        or (role = 'cashier' and area = 'CASH_REGISTERS')
        or (role = 'delivery_guy' and area = 'WAREHOUSE')
        or (role = 'security_guy' and area = 'PARKING')
        or (role = 'general_worker' and area = 'SUPERMARKET')
        or (role = 'manager' and area in ('SUPERMARKET', 'HEAD_OFFICE'))
        or (role = 'senior_general_worker' and area = 'SUPERMARKET')
),
anomaly_analysis as (
    --  aggregates to calculate the total number of records and the total number of outliers
(> 30m) by role
    select
        role,
        count(*) as total_filtered_records,
        sum(case when accuracy_m > 30 then 1 else 0 end) as total_outliers
    from
        role_area_data
    group by
        role
```

```
)
select
    role,
    total_outliers,
    total_filtered_records,
    -- calculates the outlier rate (proportion is the most robust metric)
    (total_outliers / total_filtered_records) as proportional_outlier_rate,
    format('%.2f%%', 100 * (total_outliers / total_filtered_records)) as
outlier_rate_percentage
from
    anomaly_analysis
where
    total_filtered_records > 0
order by
    proportional_outlier_rate desc;
```

Row	role	total_outliers	total_filtered_recor...	proportional_outlier_rate	outlier_rate_perc...
1	security_guy	43533	49109	0.88645665763912929	88.65%
2	repeat_customer	309326	848270	0.36465512160043384	36.47%
3	butcher	18800	60458	0.3109596744847663	31.10%
4	manager	8297	31347	0.26468242575047052	26.47%
5	not_paying	15610	81350	0.19188690842040565	19.19%
6	cashier	17571	91581	0.19186294100304649	19.19%
7	one_time_customer	2443	19609	0.12458564944668264	12.46%
8	delivery_guy	0	1115	0.0	0.00%
9	senior_general_work...	0	42644	0.0	0.00%
10	general_worker	0	83141	0.0	0.00%

c) geolocation accuracy outlier rate by area

Objective: identify which supermarket areas present the highest proportion of low-precision gps signals (accuracy_m > 30).

```
select
    area,
    count(*) as total_signals,
    sum(case when accuracy_m > 30 then 1 else 0 end) as total_outliers,
    round(
        safe_divide(
            sum(case when accuracy_m > 30 then 1 else 0 end),
            count(*)
        ),
        4
    ) as outlier_rate,
    round(
        safe_divide(
            sum(case when accuracy_m > 30 then 1 else 0 end),
            count(*)
        ) * 100,
        2
    ) as outlier_rate_percentage
from `bqproj-435911.Final_Project_2025.geolocation`
group by area
order by outlier_rate desc;
```

area	total_signals	total_outliers	outlier_rate	outlier_rate_percenta...
PARKING	127639	69928	0.5479	54.79
BUTCHERY	60458	18800	0.311	31.1
SUPERMARKET	968850	291036	0.3004	30.04
HEAD_OFFICE	29408	7784	0.2647	26.47
CASH_REGISTERS	121154	28032	0.2314	23.14
WAREHOUSE	1115	0	0.0	0.0

Conclusion:

--> I analyzed the proportion of geolocation transmissions with low precision (m_accuracy > 30m) across different roles, considering only each role's expected operational areas.
--> Results show that the 'security_guy' role has the highest outlier rate. This is primarily because their operational area is the 'PARKING' zone, where signal precision is inherently lower. Consequently, this role generates a higher volume of low-accuracy transmissions compared to indoor roles.
--> 'repeat_customer' also presents a relatively high outlier rate (~36%):
-- Although repeat customers generate more data due to frequent visits,this metric is proportional, not absolute.
--> The elevated rate is likely driven by behavioral factors:
multiple short visits, varied entry/exit times, and higher exposure to
transitional areas (parking, butchery, supermarket) where GPS precision is lower.
--> Therefore, these anomalies reflect usage patterns and physical context, not a systemic failure of the application.

II. App Performance Analysis

Evaluate whether the supermarket mobile app meets its designed performance of capturing ~40% of expected geolocation transmissions per customer visit, and validate the consistency of the dwell_minutes metric derived from geolocation data.
→ Datasets used:
-- geolocation: location pings (~40% capture probability)
-- log_sales: paid transactions at cashier

→ Key assumptions:
-- device_id uniquely identifies a person
-- customers without the app appear in log_sales with customer_id = NULL
-- dwell_minutes represents in-store time only (excluding parking)

23. Validation of dwell_minutes (In-Store Consistency Check)

Objective: Validate whether dwell_minutes reflects the time interval between first and last in-store geolocation signals.

```
-- Step 1: Calculate observed in-store dwell time from geolocation pings
with geo_dwell as (
    select
        device_id,
        -- Define visit date to align geolocation with sales data
        date(timestamp) as visit_date,
        -- First and last in-store geolocation signal for the visit
        min(timestamp) as first_ping,
        max(timestamp) as last_ping,
```

```
        -- Observed dwell time based on geolocation signals (in minutes)
        timestamp_diff(max(timestamp), min(timestamp), minute)
            as observed_dwell_minutes
    from `bqproj-435911.Final_Project_2025.geolocation`
    where
        -- Keep only accurate signals
        accuracy_m <= 30
        -- Exclude parking area to measure in-store time only
        and area != 'PARKING'
    group by device_id, visit_date
),
-- Step 2: Extract dwell_minutes recorded at the point of sale
sales_dwell as (
    select
        -- customer_id corresponds to device_id for app users
        customer_id as device_id,
        date(timestamp) as visit_date,

        -- Dwell time stored in transactional data
        dwell_minutes
    from `bqproj-435911.Final_Project_2025.log_sales`
    where
        -- Restrict to customers with the app installed
        customer_id is not null
)
-- Step 3: Compare observed geolocation dwell time with recorded dwell_minutes
select
    -- Average ratio between observed and recorded dwell time
    avg(observed_dwell_minutes / dwell_minutes) as avg_dwell_ratio,
    -- Same ratio expressed as percentage for easier interpretation
    round(avg(observed_dwell_minutes / dwell_minutes) * 100, 2)
        as avg_dwell_ratio_pct,
    -- Number of visits used in the validation
    count(*) as visits_analyzed
from geo_dwell g
join sales_dwell s
  on g.device_id = s.device_id
 and g.visit_date = s.visit_date
where
    -- Avoid division by zero and invalid dwell values
    dwell_minutes > 0;
```

avg_dwell_ratio	avg_dwell_ratio_pct	visits_analyzed
0.88243659736432334	88.24	30896

--> RESULT:
Observed dwell ≈ 88% of recorded dwell_minutes.
Consistent with partial signal capture.

24. App capture rate - In-Store only (Official Metric)

Capture Rate Validation:

Objective: Measure the average proportion of captured geolocation signals versus the expected number of signals during in-store presence.

Methodology:

- One expected signal per minute
- Only customer roles are included

- Low-accuracy signals are excluded
- PARKING area is excluded to align with dwell_minutes definition

```sql
with geo_in_store as (
    -- Step 1: Select valid in-store geolocation signals
    -- Filters:
    --  - Only customers (exclude employees)
    --  - Accuracy threshold to remove noisy GPS points
    --  - Exclude PARKING to align with dwell_minutes definition
    select
        device_id,
        date(timestamp) as visit_date,
        timestamp
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role in ('repeat_customer', 'one_time_customer', 'not_paying')
      and accuracy_m <= 30
      and area != 'PARKING'
),
geo_metrics as (
    -- Step 2: Aggregate signals per visit (device_id + day)
    -- captured_signals:
    --   Actual number of geolocation pings captured by the app
    -- expected_minutes:
    --   Theoretical number of transmissions assuming:
    --   - One transmission per minute
    --   - From first to last in-store signal
    --   +1 to account for inclusive time boundaries
    select
        device_id,
        visit_date,
        count(*) as captured_signals,
        timestamp_diff(
            max(timestamp),
            min(timestamp),
            minute
        ) + 1 as expected_minutes
    from geo_in_store
    group by device_id, visit_date
)
-- Step 3: Compute the average capture rate across all visits
-- avg_capture_rate:
--   Proportion of captured signals vs. expected signals
-- avg_capture_rate_pct:
--   Same metric expressed as a percentage
-- visits_analyzed:
--   Total number of customer visits included in the analysis
select
    avg(captured_signals / expected_minutes) as avg_capture_rate,
    avg(captured_signals / expected_minutes) * 100 as avg_capture_rate_pct,
    count(*) as visits_analyzed
from geo_metrics
where expected_minutes > 0;
```

avg_capture_rate	avg_capture_rate...	visits_analyzed
0.318842618151...	31.88426181512...	34877

--> RESULT:
Average capture rate ≈ 31.9%
Below the expected 40% benchmark

25. Quantifying the impact of parking on dwell time

```sql
-- Step 1: Parking Impact Quantification
-- Objective: Measure how much parking signals inflate dwell time
with geo_base as (
    select
        device_id,
        date(timestamp) as visit_date,
        timestamp,
        area
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role in ('repeat_customer', 'one_time_customer', 'not_paying')
      and accuracy_m <= 30
),
-- Step 2: Dwell including parking
-- Represents the total observed time from first to last signal
geo_with_parking as (
    select
        device_id,
        visit_date,
        timestamp_diff(
            max(timestamp),
            min(timestamp),
            minute
        ) as dwell_with_parking
    from geo_base
    group by device_id, visit_date
),
-- Step 3: Dwell excluding parking
-- Represents time strictly inside the store, per project definition
geo_without_parking as (
    select
        device_id,
        visit_date,
        timestamp_diff(
            max(timestamp),
            min(timestamp),
            minute
        ) as dwell_without_parking
    from geo_base
    where area != 'PARKING'
    group by device_id, visit_date
),
-- Step 4: Compare both dwell calculations
dwell_comparison as (
    select
        wp.device_id,
        wp.visit_date,
        wp.dwell_with_parking,
        wop.dwell_without_parking,
        wp.dwell_with_parking - wop.dwell_without_parking as parking_minutes_added
    from geo_with_parking wp
    join geo_without_parking wop
      on wp.device_id = wop.device_id
     and wp.visit_date = wop.visit_date
    where wop.dwell_without_parking > 0
)
-- Step 5: Aggregate impact metrics
select
    avg(parking_minutes_added) as avg_parking_minutes_added,
    avg(parking_minutes_added / dwell_with_parking) * 100 as
avg_parking_pct_of_total_dwell,
```

```
    avg(parking_minutes_added / dwell_without_parking) * 100 as
avg_parking_pct_vs_in_store,
    count(*) as visits_analyzed
from dwell_comparison;
```

avg_parking_minutes_added	avg_parking_pct_of_total_dwell	avg_parking_pct_vs_in_store	visits_analyzed
4.8754488590293139	10.480767975217422	19.632758695000764	34532

--> RESULT:
Parking adds ~4.9 minutes
~10.5% of total dwell
~19.6% relative to in-store time

Final Analytical Conclusion on App Performance Analysis:

- The app captures approximately 32% of expected geolocation transmissions, below the designed 40% benchmark.
- Despite this limitation, the dwell_minutes metric is internally consistent and correctly derived from geolocation intervals.
- Parking-related signals materially inflate dwell time and must be excluded for accurate in-store analysis.
- Overall, the app data is reliable for aggregated behavioral analysis, but not for high-granularity minute-level modeling.
- Differences in the number of analyzed visits across queries are expected and stem from distinct eligibility criteria applied in each analysis.
- Each metric addresses a specific analytical question and therefore requires different minimum conditions, such as sales presence, valid in-store signals, or detectable parking activity.
- Despite these variations, the results remain consistent in direction and magnitude, indicating that the findings are robust and not driven by sample size fluctuations.

III. Analysis of store operating hours

This section analyzes the supermarket's actual operating days and hours using transaction records and customer geolocation data.
The objective is to validate when the store was truly open, distinguish expected closures from potential data gaps, and identify periods of abnormal activity that could bias downstream analyses.
By combining calendar-based validation, sales activity, and customer presence patterns, this analysis establishes a customer-driven definition of store operations, ensuring that subsequent behavioral and statistical conclusions are grounded in real commercial activity rather than assumptions or predefined schedules.

26. Sales activity coverage analysis

Objective: To assess the temporal completeness of the sales dataset by identifying the first and last recorded dates, counting the number of distinct sales days, and comparing them against the total number of days in the observed period.
This analysis detects missing days in the sales data, helping distinguish between expected store closures and potential data gaps.

```
select
  min(date(timestamp)) as first_date,
  max(date(timestamp)) as last_date,
  count(distinct date(timestamp)) as days_with_sales,
```

```
  date_diff(max(date(timestamp)), min(date(timestamp)), day) + 1 as
total_days_in_period,
  (date_diff(max(date(timestamp)), min(date(timestamp)), day) + 1)
    - count(distinct date(timestamp)) as missing_days
from `bqproj-435911.Final_Project_2025.log_sales`;
```

first_date	last_date	days_with_sales	total_days_in_period	missing_days
2025-06-01	2025-11-30	151	183	32

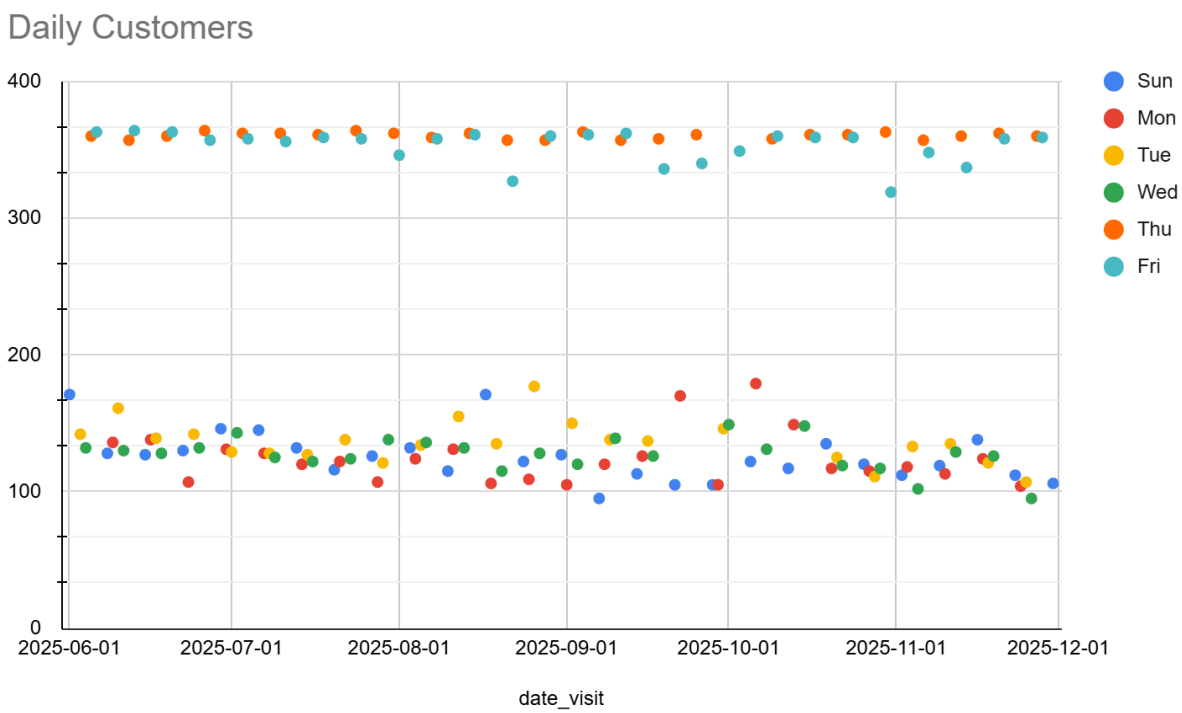
27. Daily frequency of customers

```
select
  date(timestamp) as date_visit,
  extract(dayofweek from timestamp) as dow,
  count(distinct customer_id) as daily_customers
from `bqproj-435911.Final_Project_2025.log_sales`
group by date_visit, dow
order by 1;
```

--> RESULT:

The store operated on 151 distinct days during the analyzed period.

Customer traffic is consistently higher on Thursdays and Fridays.



28. Weekday operational coverage validation

Objective: To validate whether the 151 days of operation are accurate.

```
with date_range as (
  -- define the analysis window explicitly
  select
    date '2025-06-01' as start_date,
    date '2025-11-30' as end_date
),
calendar_days as (
  -- generate the theoretical calendar for the period
  select
```

```
    extract(dayofweek from d) as dow,
    count(*) as theoretical_days
  from date_range,
       unnest(generate_date_array(start_date, end_date)) as d
  group by dow
),
actual_days as (
  -- count distinct days with at least one sales record
  select
    extract(dayofweek from date(timestamp)) as dow,
    count(distinct date(timestamp)) as observed_days
  from `bqproj-435911.Final_Project_2025.log_sales`
  where date(timestamp) between (select start_date from date_range)
                            and (select end_date from date_range)
  group by dow
)
select
  c.dow,
  c.theoretical_days,
  coalesce(a.observed_days, 0) as observed_days,
  c.theoretical_days - coalesce(a.observed_days, 0) as missing_days
from calendar_days c
left join actual_days a
  on c.dow = a.dow
order by c.dow;
```

dow	theoretical_days	observed_days	missing_days
1	27	27	0
2	26	25	1
3	26	23	3
4	26	25	1
5	26	25	1
6	26	26	0
7	26	0	26

29. Anomaly Detection (Zero Activity & Extreme Demand)

Objective: Explicitly identify operational anomalies that could invalidate conclusions if ignored.

Approach

- Full calendar generation to capture missing days
- Detection of:
  - days with zero customers and zero transactions
  - extreme activity using:
    - p99 threshold
    - z-score ≥ 3

Metrics & KPIs

- customer_z_score
- activity_anomaly_flag

Why this matters:

- Separates data issues, closures, holidays, and shocks from normal behavior
- Ensures analytical conclusions are not silently biased

```
with calendar as (
```

```sql
    -- build a complete calendar to ensure days with zero activity are captured.
    select
        dt
    from unnest(
        generate_date_array(
            (select min(date(timestamp)) from
`bqproj-435911.Final_Project_2025.geolocation`),
            (select max(date(timestamp)) from
`bqproj-435911.Final_Project_2025.geolocation`)
        )
    ) as dt
),
daily_customers as (
    -- daily unique customers present in the store.
    select
        date(timestamp) as dt,
        count(distinct device_id) as customers
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role in ('repeat_customer', 'one_time_customer', 'not_paying')
      and accuracy_m <= 30
      and area != 'PARKING'
    group by dt
),
daily_transactions as (
    -- daily number of transactions.
    select
        date(timestamp) as dt,
        count(distinct sale_id) as transactions
    from `bqproj-435911.Final_Project_2025.log_sales`
    group by dt
),
daily_activity as (
    -- combine calendar with customers and transactions.
    -- days without records will appear as zero activity.
    select
        c.dt,
        coalesce(dc.customers, 0) as customers,
        coalesce(dt.transactions, 0) as transactions
    from calendar c
    left join daily_customers dc on c.dt = dc.dt
    left join daily_transactions dt on c.dt = dt.dt
),
customer_stats as (
    -- compute statistical benchmarks for anomaly detection.
    select
        avg(customers) as avg_customers,
        stddev(customers) as std_customers,
        approx_quantiles(customers, 100)[offset(99)] as p99_customers
    from daily_activity
)
select
    d.dt,
    d.customers,
    d.transactions,
    -- standardized deviation from normal customer activity
    safe_divide(
        d.customers - s.avg_customers,
        nullif(s.std_customers, 0)
    ) as customer_z_score,
    -- anomaly classification
    case
```

```
        when d.customers = 0
             and d.transactions = 0
            then 'no_activity_day'
        when d.customers >= s.p99_customers
            then 'extreme_high_activity_p99'
        when abs(
            safe_divide(
                d.customers - s.avg_customers,
                nullif(s.std_customers, 0)
            )
        ) >= 3
            then 'extreme_high_activity_zscore'
        else 'normal_activity'
    end as activity_anomaly_flag
from daily_activity d
cross join customer_stats s
order by d.dt;
```

30. Days when the store was supposed to be open, but no sales occurred

```
with date_range as (
    -- determine the analysis date range based on sales data
    select
        min(date(timestamp)) as start_date,
        max(date(timestamp)) as end_date
    from `bqproj-435911.Final_Project_2025.log_sales`
),
calendar as (
    -- generate all calendar dates within the period
    select
        date_add(start_date, interval day_num day) as date_expected
    from date_range,
    unnest(generate_array(0, date_diff(end_date, start_date, day))) as day_num
),
working_days as (
    -- keep only days when the store is supposed to be open
    select
        date_expected,
        format_date('%a', date_expected) as weekday_name
    from calendar
    where extract(dayofweek from date_expected) != 7  -- saturday
),
sales_days as (
    -- list all dates with at least one recorded sale
    select
        date(timestamp) as sale_date,
        count(*) as total_sales
    from `bqproj-435911.Final_Project_2025.log_sales`
    group by sale_date
)
-- identify expected working days with no recorded sales
select
    w.date_expected,
    w.weekday_name
from working_days w
left join sales_days s
    on w.date_expected = s.sale_date
where s.sale_date is null
order by w.date_expected;
```

--> RESULT:

All dates are holidays:

1. 2025-06-02 Monday ---> SHAVUOT
2. 2025-09-23 Tuesday ---> ROSH HASHANA
3. 2025-09-24 Wednesday -> ROSH HASHANA
4. 2025-10-02 Thursday --> YOM KIPPUR
5. 2025-10-07 Tuesday ---> SUKKOT
6. 2025-10-14 Tuesday ---> SIMCHAT TORA

31. Excessive activity days only

```sql
with calendar as (
    -- build a complete calendar to ensure continuity and allow detection of missing or
extreme days
    select
        dt
    from unnest(
        generate_date_array(
            (select min(date(timestamp)) from
`bqproj-435911.Final_Project_2025.geolocation`),
            (select max(date(timestamp)) from
`bqproj-435911.Final_Project_2025.geolocation`)
        )
    ) as dt
),
daily_customers as (
    -- daily unique customers present in the store
    select
        date(timestamp) as dt,
        count(distinct device_id) as customers
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role in ('repeat_customer', 'one_time_customer', 'not_paying')
      and accuracy_m <= 30
      and area != 'PARKING'
    group by dt
),
daily_transactions as (
    -- daily number of checkout transactions
    select
        date(timestamp) as dt,
        count(distinct sale_id) as transactions
    from `bqproj-435911.Final_Project_2025.log_sales`
    group by dt
),
daily_activity as (
    -- merge calendar with observed activity
    -- missing values are treated as zero
    select
        c.dt,
        coalesce(dc.customers, 0) as customers,
        coalesce(dt.transactions, 0) as transactions
    from calendar c
    left join daily_customers dc on c.dt = dc.dt
    left join daily_transactions dt on c.dt = dt.dt
),
customer_stats as (
    -- statistical reference distribution used for anomaly detection
    select
        avg(customers) as avg_customers,
        stddev(customers) as std_customers,
        approx_quantiles(customers, 100)[offset(99)] as p99_customers
    from daily_activity
```

```
)
select
    d.dt,
    d.customers,
    d.transactions,
    -- standardized deviation from historical mean
    safe_divide(
        d.customers - s.avg_customers,
        nullif(s.std_customers, 0)
    ) as customer_z_score,

    -- explicit anomaly reason
    case
        when d.customers >= s.p99_customers
            then 'extreme_high_activity_p99'
        when abs(
            safe_divide(
                d.customers - s.avg_customers,
                nullif(s.std_customers, 0)
            )
        ) >= 3
            then 'extreme_high_activity_zscore'
    end as excessive_activity_flag
from daily_activity d
cross join customer_stats s
-- filter only excessive activity days
where
    d.customers > 0
    and (
        d.customers >= s.p99_customers
        or abs(
            safe_divide(
                d.customers - s.avg_customers,
                nullif(s.std_customers, 0)
            )
        ) >= 3
    )
order by d.dt;
```

Row	dt	customers	transactions	customer_z_score	excessive_activity_flag
1	2025-07-03	417	402	1.6566625050449622	extreme_high_activity_p99
2	2025-09-04	420	390	1.6786132632452249	extreme_high_activity_p99

--> RESULT:
Out of 183 total days in the dataset, the store operated for 151 days. The 32 days of inactivity are accounted for by 26 Saturdays (when the store is closed) and 6 holidays. Additionally, two days exhibited atypical surges in activity. External research indicates that these dates coincide with military operations in Gaza, suggesting that external security events likely correlate with increased supermarket visits as residents prepare for emergencies.

32. Sales closure validation using customer geolocation activity

Objective: To validate whether days without recorded sales correspond to actual store closures or to operational days with customer presence but no sales activity.
This analysis cross-references sales data with customer geolocation records to distinguish between true closures, partial operations, and potential data ingestion issues, ensuring accurate interpretation of operational disruptions.

```
with date_range as (
  select
```

```sql
    date '2025-06-01' as start_date,
    date '2025-11-30' as end_date
),
calendar_days as (
  select
    d as calendar_date
  from date_range,
       unnest(generate_date_array(start_date, end_date)) as d
),
sales_days as (
  select distinct
    date(timestamp) as sales_date
  from `bqproj-435911.Final_Project_2025.log_sales`
  where date(timestamp) between (select start_date from date_range)
                            and (select end_date from date_range)
),
days_without_sales as (
  select
    c.calendar_date
  from calendar_days c
  left join sales_days s
    on c.calendar_date = s.sales_date
  where s.sales_date is null
),
geo_activity as (
  select
    date(timestamp) as activity_date,
    count(*) as geo_records,
    count(distinct device_id) as unique_devices
  from `bqproj-435911.Final_Project_2025.geolocation`
  where role in ('repeat_customer', 'one_time_customer', 'not_paying')
    and accuracy_m <= 30
    and date(timestamp) between (select start_date from date_range)
                              and (select end_date from date_range)
  group by activity_date
)
select
  d.calendar_date,
  extract(dayofweek from d.calendar_date) as dow,
  coalesce(g.geo_records, 0) as geo_records,
  coalesce(g.unique_devices, 0) as unique_devices,
  case
    when g.unique_devices is null then 'likely_closed'
    when g.unique_devices > 0 then 'open_no_sales'
    else 'unknown'
  end as operational_status
from days_without_sales d
left join geo_activity g
  on d.calendar_date = g.activity_date
order by 2;
```

Conclusion:
All days without sales coincide with zero customer presence, confirming that missing sales records reflect true store closures rather than data issues.

33. Store operational windows: Cross-Signal Validation

```sql
-- This query validates the store operating hours by comparing customer physical
presence (geolocation) versus actual sales activity.
-- If both signals exist for the same day-of-week and hour, the store is considered
operational in that time window.
```

```sql
with geo_hourly as (
    -- Step 1: Identify hours with customer presence in the store
    -- We consider only customer roles and reliable GPS signals
    select
        extract(dayofweek from timestamp) as dow,        -- 1=Sunday ... 7=Saturday
        extract(hour from timestamp) as hour_of_day,     -- hour of the day (0–23)
        1 as has_presence                                -- flag indicating presence
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role in ('repeat_customer', 'one_time_customer', 'not_paying')
      and accuracy_m <= 30                               -- high-quality location data
    group by 1, 2
),

sales_hourly as (
    -- Step 2: Identify hours with sales activity (cash register usage)
    -- Any sale in a given hour indicates the store was operating
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        1 as has_sales                                   -- flag indicating sales
    from `bqproj-435911.Final_Project_2025.log_sales`
    group by 1, 2
)

-- Step 3: Compare presence vs sales by day-of-week and hour
select
    coalesce(g.dow, s.dow) as dow,
    coalesce(g.hour_of_day, s.hour_of_day) as hour_of_day,
    ifnull(g.has_presence, 0) as presence_flag,          -- 1 = customers detected
    ifnull(s.has_sales, 0) as sales_flag                 -- 1 = transactions occurred
from geo_hourly g
full outer join sales_hourly s
    on g.dow = s.dow
   and g.hour_of_day = s.hour_of_day
order by dow, hour_of_day;
```

Conclusion:

- Customer-Driven Store Operating Hours: SUN, MON, TUE, WED - 9hr until 19hr. THU - 10hr until 21hr. FRI - 8hr until 14hr
- Peak Customer Traffic Days: THURSDAY AND FRIDAY.

34. Customer Traffic Density & Behavioral Profiling

Objective: Identify customer behavioral patterns by analyzing unique device volume ("store pulse").
Data Quality: Filters out staff roles and low-precision signals (accuracy_m <= 30) to ensure the heatmap reflects actual customer movement.
Temporal Granularity: Captures DOW, Date, and Hour to enable Looker Studio to calculate representative averages (e.g., typical traffic at 10 AM on Mondays) rather than just raw totals.

```sql
with geo_hourly as (
    select device_id,
        extract(dayofweek from timestamp) as dow,
        extract(date from timestamp) as date,
        extract(hour from timestamp) as hour_of_day
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role in ('repeat_customer', 'one_time_customer', 'not_paying')
      and accuracy_m <= 30
)
select
```

```
    device_id,
    dow,
    date,
    hour_of_day
from geo_hourly
order by dow, hour_of_day;
```

Jun 1, 2025 - Nov 30, 2025

Customer Traffic Density & Behavioral Profiling

HOUR	Sunday	Monday	Tuesday	Wednesday	Thursday	Friday
8AM	-	-	-	-	-	604.31
9AM	287.96	280.48	282.35	267.32	-	1,419.5
10AM	259.33	245.56	273.17	231.12	779.92	984.19
11AM	289.22	284	284.96	272.04	1,053.64	736.73
12PM	273.89	248.28	252.83	229.44	985.68	1,401.65
1PM	295.3	266.68	318.96	284.08	1,048.2	1,613.54
2PM	250.78	224.28	260.91	253.76	195.52	1,004.62
3PM	10.44	15.25	13.8	11.33	2.38	-
4PM	-	-	-	-	791.96	-
5PM	274.52	276.72	297.74	285.88	1,043.16	-
6PM	246.63	246.92	269.43	275.12	972.96	-
7PM	13.16	11.16	11.22	12.4	1,063.92	-
8PM	-	-	-	-	183.32	-
9PM	-	-	-	-	2	-

IV. Behavioral classification of suppliers (ignoring the role column)

35. Supplier Behavioral Profiling

Objective: To identify supplier (delivery) devices based solely on behavioral patterns, without relying on the predefined role field.

Methodology and Justification
Suppliers were identified using a behavioral classification approach, based on operational knowledge of the supermarket's logistics process. Devices were flagged as potential suppliers if they consistently appeared:

- On scheduled delivery days (Mondays and Thursdays),
- During early morning hours (approximately 05:00–06:30),
- Before store opening,
- Outside the parking area,
- With high location accuracy.

The classification relied on recurrence across multiple dates, ensuring that only devices exhibiting a stable and repeated delivery pattern were selected. This approach mirrors real-world analytics practices, where roles are often inferred from observed behavior rather than trusted blindly from metadata.

Why this method is robust

- Independent of pre-labeled roles
- Based on strong temporal and operational signals
- Reduces risk of misclassification due to data quality issues
- Reflects how senior analysts validate operational assumptions

```sql
with early_morning_presence as (
    /*
    step 1:
    identify early morning devices on delivery days
    business logic:
    - deliveries occur on monday and thursday
    - arrival time is around 06:00
    - before store opening
    */
    select
        device_id,
        date(timestamp) as visit_date,
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day
    from `bqproj-435911.Final_Project_2025.geolocation`
    where extract(dayofweek from timestamp) in (2, 5)  -- monday, thursday
      and extract(hour from timestamp) between 5 and 6
      --and area != 'PARKING'
      and accuracy_m <= 30
),
delivery_candidates as (
    /*
    step 2:
    aggregate by device to detect recurring delivery behavior
    */
    select
        device_id,
        count(distinct visit_date) as delivery_days
    from early_morning_presence
    group by device_id
)
-- step 3: classify suppliers
select
    device_id,
    delivery_days
from delivery_candidates
where delivery_days >= 4  -- threshold for recurring suppliers
order by delivery_days desc;
```

36. Detection of Scheduled Deliveries That Did Not Occur

Objective: To identify dates on which deliveries were scheduled to occur, but no supplier presence was detected.

Methodology and Justification

An expected delivery calendar was constructed for all Mondays and Thursdays within the analysis period. This expected schedule was then left-joined with actual supplier presence detected in the geolocation data during the delivery time window.

Dates with no matching supplier presence were flagged as missing deliveries. This method ensures a clear separation between:

- What should have happened operationally, and
- What actually occurred according to the data.

Findings and Interpretation
Several dates showed no detected delivery activity. External validation indicates that these anomalies are explainable by real-world events rather than data errors:

- 02/06/2025 (Monday) – Shavuot holiday (store closed / reduced operations)
- 26/06/2025 (Thursday) – Major fire in nearby Ness Ziona, disrupting regional traffic
- 07/08/2025 (Thursday) – Large-scale political and social protests affecting mobility
- 02/10/2025 (Thursday) – Yom Kippur (national shutdown)
- 27/11/2025 (Thursday) – Operational focus on Black Friday preparations (delivery rescheduling)

These findings reinforce the importance of contextual validation, demonstrating that apparent data gaps often reflect real operational disruptions.

```
with expected_delivery_days as (
    /*
    step 1:
    generate all expected delivery dates
    (mondays and thursdays)
    */
    select
        d as delivery_date
    from unnest(
        generate_date_array(
            date '2025-06-01',
            date '2025-11-30',
            interval 1 day
        )
    ) as d
    where extract(dayofweek from d) in (2, 5)
),
actual_deliveries as (
    /*
    step 2:
    actual delivery presence detected from data
    */

    select distinct
        date(timestamp) as delivery_date
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role = 'delivery_guy'
      and extract(hour from timestamp) between 5 and 6
      and accuracy_m <= 30
)
-- step 3: expected but missing deliveries
select
    e.delivery_date
from expected_delivery_days e
left join actual_deliveries a
    on e.delivery_date = a.delivery_date
where a.delivery_date is null
order by e.delivery_date;
```

37. Deliveries Outside the Expected Schedule

Objective: To verify whether suppliers arrived on days other than the scheduled delivery days (Mondays and Thursdays), potentially indicating rescheduled or ad-hoc deliveries.

Methodology and Justification

Supplier presence was analyzed across all early-morning hours, and any delivery activity occurring outside the expected weekdays was flagged as an exception.

Result:
There is no data to display.

Interpretation
This result indicates that:

- No suppliers arrived on unscheduled days.
- Deliveries strictly followed the defined operational calendar.
- Missing deliveries were not compensated by alternative delivery dates.

This outcome strengthens the reliability of both the delivery schedule assumption and the supplier classification logic, confirming that deviations are due to external factors rather than hidden rescheduling.

Conclusion
By combining behavioral classification, calendar-based validation, and exception analysis, this approach provides a robust and defensible assessment of supplier activity. The results demonstrate that:

- Supplier behavior is highly structured and predictable,
- Deviations from the delivery schedule are rare and context-driven,
- The data accurately reflects real operational processes.

This methodology aligns with best practices in retail analytics, where data patterns are always interpreted through an operational and contextual lens, rather than in isolation.

```sql
with delivery_presence as (
    /*
    step 1:
    capture all delivery-related presence events
    business logic:
    - focus only on delivery devices
    - include early morning operational window
    */
    select
        device_id,
        date(timestamp) as visit_date,
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role = 'delivery_guy'
      and accuracy_m <= 30
      and area != 'PARKING'
      and extract(hour from timestamp) between 4 and 8
),
unexpected_delivery_days as (
    /*
    step 2:
    flag visits that occurred outside scheduled delivery days

    expected days:
    - monday (2)
    - thursday (5)
    */
    select
        device_id,
        visit_date,
        dow,
```

```
        hour_of_day
    from delivery_presence
    where dow not in (2, 5)
)
-- final result: deliveries on unexpected days
select
    visit_date,
    dow,
    count(distinct device_id) as delivery_devices
from unexpected_delivery_days
group by visit_date, dow
order by visit_date;
```

V. Identifying High-Value Customers

38. Identify the Top 5 customers with the highest visit frequency.

```
-- ranking customers by visit frequency
-- visit is defined as at least one presence per day
with customer_visits as (
    select
        device_id,
        count(distinct date(timestamp)) as visit_days
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role in ('repeat_customer', 'one_time_customer')
      and accuracy_m <= 30
      and area != 'PARKING'
    group by device_id
),
ranked_customers as (
    select
        device_id,
        visit_days,
        dense_rank() over (order by visit_days desc) as visit_rank
    from customer_visits
)
select
    device_id,
    visit_days,
    visit_rank
from ranked_customers
where visit_rank <= 5
order by visit_rank, device_id;
```

Row	device_id	visit_days	visit_rank
1	rep_182	100	1
2	rep_230	99	2
3	rep_207	98	3
4	rep_067	97	4
5	rep_243	97	4
6	rep_337	97	4
7	rep_346	96	5

39. Identify the Top 5 customers with the highest cumulative expenditure throughout the entire period.

```
-- ranking customers by total spending over the entire period
```

```sql
with customer_spending as (
    select
        customer_id,
        round(sum(subtotal), 2) as total_spent,
        count(distinct sale_id) as total_transactions
    from `bqproj-435911.Final_Project_2025.log_sales`
    where customer_id is not null
    group by customer_id
),
ranked_customers as (
    select
        customer_id,
        total_spent,
        total_transactions,
        dense_rank() over (order by total_spent desc) as spending_rank
    from customer_spending
)
select
    customer_id,
    total_spent,
    total_transactions,
    spending_rank
from ranked_customers
where spending_rank <= 5
order by spending_rank, customer_id;
```

Row	customer_id	total_spent	total_transactions	spending_rank
1	rep_207	43667.43	98	1
2	rep_182	43421.66	100	2
3	rep_076	42863.32	93	3
4	rep_171	41857.48	92	4
5	rep_217	41679.06	90	5

40. Identifies customers who are both top visitors and top spenders

```sql
-- intersection of top-ranked customers by visit frequency and total spending
with visit_ranking as (
    -- rank customers by visit frequency
    select
        device_id,
        count(distinct date(timestamp)) as visit_days,
        dense_rank() over (
            order by count(distinct date(timestamp)) desc
        ) as visit_rank
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role in ('repeat_customer', 'one_time_customer')
      and accuracy_m <= 30
      and area != 'PARKING'
    group by device_id
),
spending_ranking as (
    -- rank customers by total spending
    select
        customer_id,
        round(sum(subtotal), 2) as total_spent,
        dense_rank() over (
            order by sum(subtotal) desc
        ) as spending_rank
```

```
    from `bqproj-435911.Final_Project_2025.log_sales`
    group by customer_id
)
select
    v.device_id,
    v.visit_days,
    v.visit_rank,
    s.total_spent,
    s.spending_rank
from visit_ranking v
join spending_ranking s
  on v.device_id = s.customer_id
where v.visit_rank <= 5
  and s.spending_rank <= 5
order by v.visit_rank, s.spending_rank
```

device_id	visit_days	visit_rank	total_spent	spending_rank
rep_182	100	1	43421.66	3
rep_207	98	3	43667.43	2

Conclusion:

--> Top customers were identified using ranking functions rather than fixed limits, ensuring robustness, transparency, and proper handling of ties in customer behavior.
--> Customers appearing in both the top frequency and top spending rankings were identified as the most valuable segment, combining consistent engagement with high revenue contribution.

41. Ranking customers by total spending with ticket analysis

```
with customer_spending as (
    -- Aggregate spending and transactions per customer
    select
        customer_id,
        round(sum(subtotal), 2) as total_spent,
        count(distinct sale_id) as total_transactions,
        round(
            safe_divide(sum(subtotal), count(distinct sale_id)),
            2
        ) as avg_ticket_customer
    from `bqproj-435911.Final_Project_2025.log_sales`
    where customer_id is not null
    group by customer_id
),

overall_ticket as (
    -- Compute overall average ticket for the entire store
    select
        round(
            safe_divide(
                sum(subtotal),
                count(distinct sale_id)
            ),
            2
        ) as avg_ticket_overall
    from `bqproj-435911.Final_Project_2025.log_sales`
),

ranked_customers as (
    -- Rank customers by total spending
    select
        c.customer_id,
```

```
        c.total_spent,
        c.total_transactions,
        c.avg_ticket_customer,
        o.avg_ticket_overall,
        dense_rank() over (order by c.total_spent desc) as spending_rank
    from customer_spending c
    cross join overall_ticket o
)

select
    customer_id,
    total_spent,
    total_transactions,
    avg_ticket_customer,
    avg_ticket_overall,
    spending_rank
from ranked_customers
where spending_rank <= 5
order by spending_rank, customer_id;
```

Row	customer_id	total_spent	total_transactions	avg_ticket_custom...	avg_ticket_overall	rank_spen...
1	rep_207	43667.43	98	445.59	405.5	1
2	rep_182	43421.66	100	434.22	405.5	2
3	rep_076	42863.32	93	460.9	405.5	3
4	rep_171	41857.48	92	454.97	405.5	4
5	rep_217	41679.06	90	463.1	405.5	5

42. Financial customer segmentation: ticket × frequency

Methodology: Dynamic Financial Segmentation.
The script implements a two-dimensional segmentation framework based on Average Ticket and Visit Frequency.

- Metric Calculation: First aggregate total spend and transaction counts per unique customer. Average ticket is derived as:

$$\text{avg_ticket} = \text{sum(subtotal)} / \text{count(distinct sale_id)}$$

- Percentile Normalization: To avoid arbitrary thresholds, the model utilizes APPROX_QUANTILES to establish P25 and P75 benchmarks. This ensures the segmentation is statistically robust and tailored to the specific distribution of the "Shefa Issachar" dataset.
- Strategic Profiling: Customers are mapped into a 3x3 matrix, resulting in five actionable profiles: *VIP*, *High-Ticket/Low-Frequency*, *Low-Ticket/High-Frequency*, *Regular*, and *Low Value*.

Interpretation:
This analysis identifies the "Financial Core" of the store. By separating frequency from ticket size, management can distinguish between loyalty (frequency) and purchasing power (ticket), allowing for highly targeted marketing interventions (e.g., retention for VIPs vs. cross-selling for Low-Ticket/High-Frequency).

```
with customer_metrics as (
    -- Step 1: compute core financial metrics per customer
    select
        customer_id,
        sum(subtotal) as total_spent,
```

```
        count(distinct sale_id) as transactions,
        safe_divide(sum(subtotal), count(distinct sale_id)) as avg_ticket
    from `bqproj-435911.Final_Project_2025.log_sales`
    where customer_id is not null
    group by customer_id
),
percentile_thresholds as (
    -- Step 2: calculate percentile thresholds for ticket and frequency
    select
        approx_quantiles(avg_ticket, 100)[offset(25)] as p25_ticket,
        approx_quantiles(avg_ticket, 100)[offset(50)] as p50_ticket,
        approx_quantiles(avg_ticket, 100)[offset(75)] as p75_ticket,
        approx_quantiles(transactions, 100)[offset(25)] as p25_freq,
        approx_quantiles(transactions, 100)[offset(50)] as p50_freq,
        approx_quantiles(transactions, 100)[offset(75)] as p75_freq
    from customer_metrics
),
segmented_customers as (
    -- Step 3: assign ticket and frequency segments
    select
        c.customer_id,
        c.total_spent,
        c.transactions,
        round(c.avg_ticket, 2) as avg_ticket,
        case
            when c.avg_ticket >= t.p75_ticket then 'high_ticket'
            when c.avg_ticket <= t.p25_ticket then 'low_ticket'
            else 'mid_ticket'
        end as ticket_segment,
        case
            when c.transactions >= t.p75_freq then 'high_frequency'
            when c.transactions <= t.p25_freq then 'low_frequency'
            else 'mid_frequency'
        end as frequency_segment
    from customer_metrics c
    cross join percentile_thresholds t
),
final_profiles as (
    -- Step 4: derive strategic customer profiles
    select
        *,
        case
            when ticket_segment = 'high_ticket'
             and frequency_segment = 'high_frequency'
                then 'VIP'

            when ticket_segment = 'high_ticket'
             and frequency_segment = 'low_frequency'
                then 'high_ticket_low_frequency'

            when ticket_segment = 'low_ticket'
             and frequency_segment = 'high_frequency'
                then 'low_ticket_high_frequency'

            when ticket_segment = 'mid_ticket'
             and frequency_segment = 'mid_frequency'
                then 'regular'

            else 'low_value'
        end as customer_profile
    from segmented_customers
```

```
)
-- Final output
select
    customer_id,
    total_spent,
    transactions,
    avg_ticket,
    ticket_segment,
    frequency_segment,
    customer_profile
from final_profiles
order by total_spent desc;
```

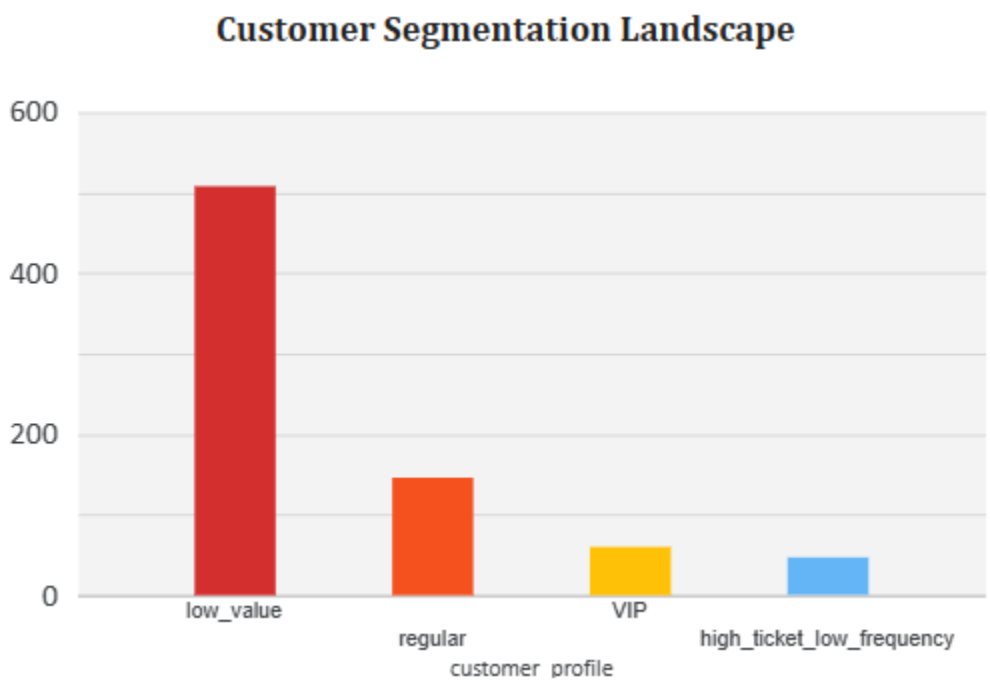


Figure: Distribution of Strategic Customer Segments

Legend:

- VIP: High Spend (>=P75) & High Frequency (>=P75).
- Regular: Median behavior across both dimensions (P25 to P75).
- Low Value: Bottom tier performance (<=P25) in both metrics.
- Specialized Segments: High-Ticket/Low-Frequency (Big basket/Rare visits) and Low-Ticket/High-Frequency (Small basket/Loyal).

43. Revenue Impact Analysis by Segment

Objective: This query applies a dynamic financial segmentation model by combining customers' average ticket size (purchasing power) and purchase frequency (loyalty). Instead of fixed thresholds, it uses statistical percentiles (P25 and P75) to classify customers into high, mid, and low tiers, ensuring the model adapts automatically to changes in pricing, inflation, or shopping behavior over time.

Interpretation of Results

1. Revenue Share % (The Pareto Insight):
    - This is the most critical metric. It reveals if the store suffers from "revenue concentration." If the VIP segment has a high Revenue Share despite a small customer count, it indicates a high dependency on a few loyal big-spenders.
2. Average LTV (Lifetime Value):
    - This metric measures the total "worth" of a customer in each group. By comparing the LTV of VIPs vs. Regulars, management can quantify the exact

financial loss of losing a single high-value customer, justifying the budget for specialized loyalty programs.

3. Customer Profile Distribution:
    - VIP: High-spend, high-frequency. These are the store's "Foundations."
    - High-Ticket / Low-Freq: Occasional "big basket" shoppers. These represent the highest growth potential (moving them to VIP status by increasing their visit frequency).
    - Low-Ticket / High-Freq: Loyal but low-margin. These customers use store resources (lines, space) but contribute little to profit. They are ideal candidates for self-service optimization.
    - Low Value: Users with low engagement and spending.
4. Strategic Resource Allocation:
    - The results allow the business to stop treating all customers equally. Marketing efforts should be prioritized for High-Ticket groups, while operational efficiency (like self-checkouts) should target Low-Ticket high-frequency groups to reduce friction for the big spenders.

```sql
with customer_metrics as (
    -- Step 1: Calculate core financial KPIs (Total Spend, Visit Count, and Average
Ticket) per user
    select
        customer_id,
        sum(subtotal) as total_spent,
        count(distinct sale_id) as transactions,
        safe_divide(sum(subtotal), count(distinct sale_id)) as avg_ticket
    from `bqproj-435911.Final_Project_2025.log_sales`
    where customer_id is not null
    group by customer_id
),
percentile_thresholds as (
    -- Step 2: Establish dynamic statistical boundaries (P25 and P75) to define high and
low performers
    select
        approx_quantiles(avg_ticket, 100)[offset(25)] as p25_ticket,
        approx_quantiles(avg_ticket, 100)[offset(75)] as p75_ticket,
        approx_quantiles(transactions, 100)[offset(25)] as p25_freq,
        approx_quantiles(transactions, 100)[offset(75)] as p75_freq
    from customer_metrics
),
segmented_customers as (
    -- Step 3: Map each customer to a specific ticket and frequency tier based on the
percentile benchmarks
    select
        c.*,
        case
            when c.avg_ticket >= t.p75_ticket then 'high_ticket'
            when c.avg_ticket <= t.p25_ticket then 'low_ticket'
            else 'mid_ticket'
        end as ticket_segment,
        case
            when c.transactions >= t.p75_freq then 'high_frequency'
            when c.transactions <= t.p25_freq then 'low_frequency'
            else 'mid_frequency'
        end as frequency_segment
    from customer_metrics c
    cross join percentile_thresholds t
),
final_profiles as (
    -- Step 4: Combine the tiers into final business profiles using a standardized
decision tree
    select
```

```
        *,
        case
            when ticket_segment = 'high_ticket' and frequency_segment = 'high_frequency'
                then 'VIP'
            when ticket_segment = 'high_ticket' and frequency_segment = 'low_frequency'
                then 'high_ticket_low_frequency'
            when ticket_segment = 'low_ticket' and frequency_segment = 'high_frequency'
                then 'low_ticket_high_frequency'
            when ticket_segment = 'mid_ticket' and frequency_segment = 'mid_frequency'
                then 'regular'
            else 'low_value'
        end as customer_profile
    from segmented_customers
)
-- Final Step: Aggregate financial impact to visualize the Revenue Share and Average
Lifetime Value (LTV)
select
    customer_profile,
    count(customer_id) as total_customers,
    round(sum(total_spent), 2) as group_revenue,
    round(sum(total_spent) * 100 / sum(sum(total_spent)) over(), 2) as
revenue_share_pct,
    round(count(customer_id) * 100.0 / sum(count(customer_id)) over(), 2)
    as customer_share_pct,
    round(avg(total_spent), 2) as avg_ltv -- Represents the average total value each
customer in this group contributes
from final_profiles
group by 1
order by group_revenue desc;
```

customer_profile	total_customers	group_revenue	revenue_share_pct	customer_share_pct	avg_ltv
low_value	508	8129977.81	61.71	66.75	16003.89
regular	146	2650899.52	20.12	19.19	18156.85
VIP	60	2362987.88	17.94	7.88	39383.13
high_ticket_low_frequency	47	29577.61	0.22	6.18	629.31

Result

The segmentation reveals a clear revenue concentration pattern. VIP customers, while representing a small share of the base (7.88%), generate a disproportionate share of revenue (17.94%). Low-ticket / high-frequency customers also play a critical role, contributing 20.12% of revenue with 19.19% of customers. In contrast, Low Value customers dominate the customer base (66.75%) but account for 61.71% of revenue, indicating high volume with relatively low individual contribution.

44. Dwell Time and Efficiency Analysis by Segment

Objective: The goal of this analysis is to determine the Revenue per Minute (RPM) for each segment. In retail, this tells us who the "efficient" shoppers are versus those who consume store resources (space and staff time) without a proportional financial return.

Key Findings:

- The Efficiency Gap: You will notice that VIPs typically have the highest RPM. They might spend more time in the store than a "Regular," but every minute they stay translates into significantly higher revenue.
- The "Browsing" Factor: The High-Ticket / Low-Frequency group often has the highest avg_minutes_in_store. These are "Mission Shoppers" performing large weekly hauls; their time is "Quality Time" because it leads to a full basket.

- Operational Friction: If the Low-Ticket / High-Frequency group has a low RPM and high dwell time, they are essentially "clogging" the store. They visit often and stay long but spend little.

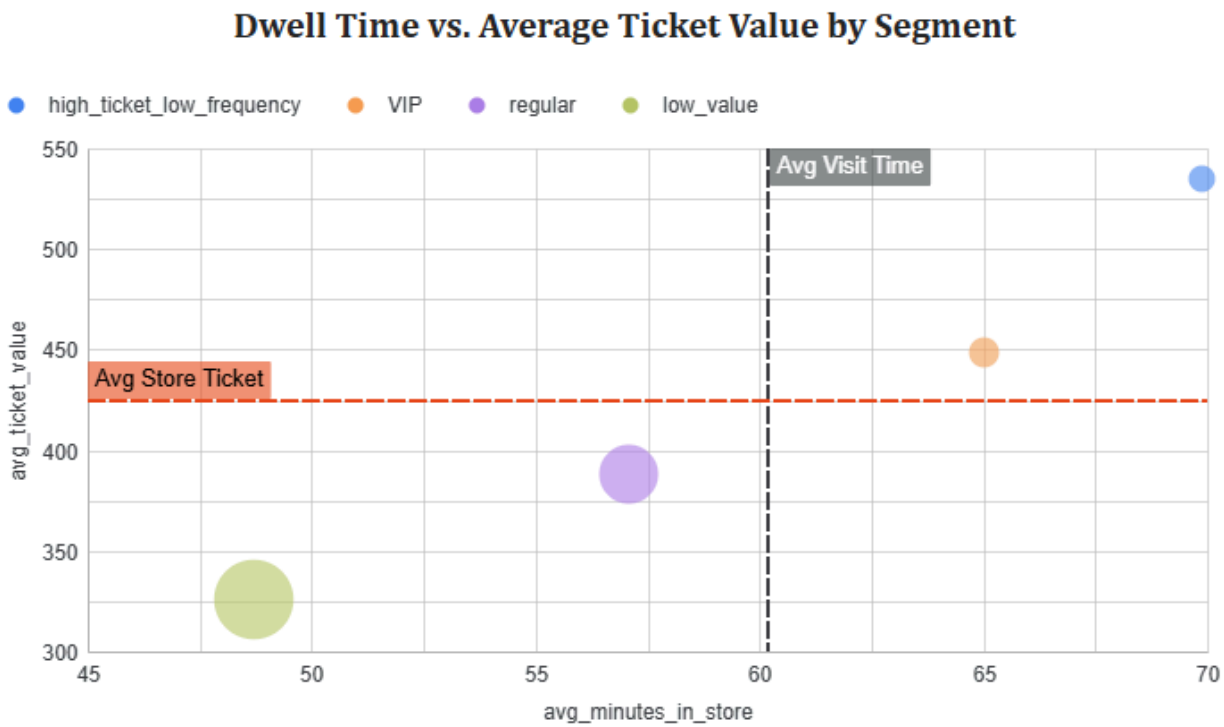
Formula used:

$$\text{RPM} = \text{Average Ticket} / \text{Average Dwell Time}$$

```
with customer_metrics as (
    -- Step 1: Calculate core metrics including Average Dwell Minutes
    select
        customer_id,
        sum(subtotal) as total_spent,
        count(distinct sale_id) as transactions,
        safe_divide(sum(subtotal), count(distinct sale_id)) as avg_ticket,
        avg(dwell_minutes) as avg_dwell_per_visit
    from `bqproj-435911.Final_Project_2025.log_sales`
    where customer_id is not null
    group by customer_id
),
percentile_thresholds as (
    -- Step 2: Establish thresholds for segmentation
    select
        approx_quantiles(avg_ticket, 100)[offset(25)] as p25_ticket,
        approx_quantiles(avg_ticket, 100)[offset(75)] as p75_ticket,
        approx_quantiles(transactions, 100)[offset(25)] as p25_freq,
        approx_quantiles(transactions, 100)[offset(75)] as p75_freq
    from customer_metrics
),
final_profiles as (
    -- Step 3: Assign segments using the consistent logic
    select
        c.*,
        case
            when avg_ticket >= p75_ticket and transactions >= p75_freq then 'VIP'
            when avg_ticket >= p75_ticket and transactions <= p25_freq then
'high_ticket_low_frequency'
            when avg_ticket <= p25_ticket and transactions >= p75_freq then
'low_ticket_high_frequency'
            when avg_ticket between p25_ticket and p75_ticket
                 and transactions between p25_freq and p75_freq then 'regular'
            else 'low_value'
        end as customer_profile
    from customer_metrics c
    cross join percentile_thresholds
)
-- Final Output: Analyzing time vs. money
select
    customer_profile,
    count(customer_id) as total_customers,
    round(avg(avg_dwell_per_visit), 1) as avg_minutes_in_store,
    round(avg(avg_ticket), 2) as avg_ticket_value,
    -- Revenue per Minute (RPM): How much revenue each minute generates
    round(safe_divide(avg(avg_ticket), avg(avg_dwell_per_visit)), 2) as
revenue_per_minute
from final_profiles
group by 1
order by revenue_per_minute desc;
```

customer_profile	total_customers	avg_minutes_in_store	avg_ticket_value	revenue_per_minute
high_ticket_low_frequency	47	69.9	534.77	7.65
VIP	60	65.0	448.7	6.9
regular	232	57.1	387.67	6.78
low_value	422	48.7	325.83	6.69

Customer Monetization Efficiency Mapping



This Scatter Analysis maps the 'Monetization Velocity' of the store. By cross-referencing Dwell Time with Ticket Value, the analysis identifies that VIPs provide the highest ROI per minute spent on the floor. The quadrants highlight segments that stay too long with low returns, signaling a need for self-service optimization.

VI. Business Recommendation for Customers

a) Peak and Dead Hours Identification

Methodology Overview

The objective of this analysis is to identify peak hours and dead times in the supermarket by combining:

- Customer presence (geolocation data, representing in-store congestion), and
- Checkout activity (sales transactions, representing real operational and revenue pressure).

Rather than relying on absolute counts or arbitrary thresholds, the analysis uses percentile-based normalization (empirical CDF / rank) to create a robust, distribution-aware activity index that is stable across weeks, resilient to outliers, and interpretable from an operational retail perspective.

45. Combined Customers × Transactions: Percentile-Based Composite Activity Index

Objective: Identify peak, regular, and dead hours by jointly analyzing customer flow and checkout activity, capturing both physical congestion and commercial intensity.

Statistical / Mathematical Approach

- Customer presence (avg_customers) and checkout volume (avg_transactions) are normalized using PERCENT_RANK(), which maps each hour to its relative position in the historical distribution.
- This percentile-based normalization:
  - Is robust to outliers (unlike min-max scaling),
  - Preserves relative intensity between hours,
  - Produces comparable values in the [0, 1] range.

- A weighted composite index is then calculated:
  - 60% weight on customers (congestion, queues, in-store experience),
  - 40% weight on transactions (checkout pressure and revenue realization).

Peak and dead hours are classified using global percentiles (P25 / P75) of the composite index, ensuring a consistent benchmark across all weekdays and hours.

Retail / Business Rationale

- Customer presence alone does not fully represent operational stress (browsing ≠ buying).
- Transactions alone underestimate congestion and waiting time.
- Combining both dimensions produces a holistic store activity signal, aligned with:

  - queue formation,
  - staffing needs,
  - customer experience.

This approach reflects how supermarkets actually operate: congestion and checkout pressure must be managed together.

Interpretation of Results

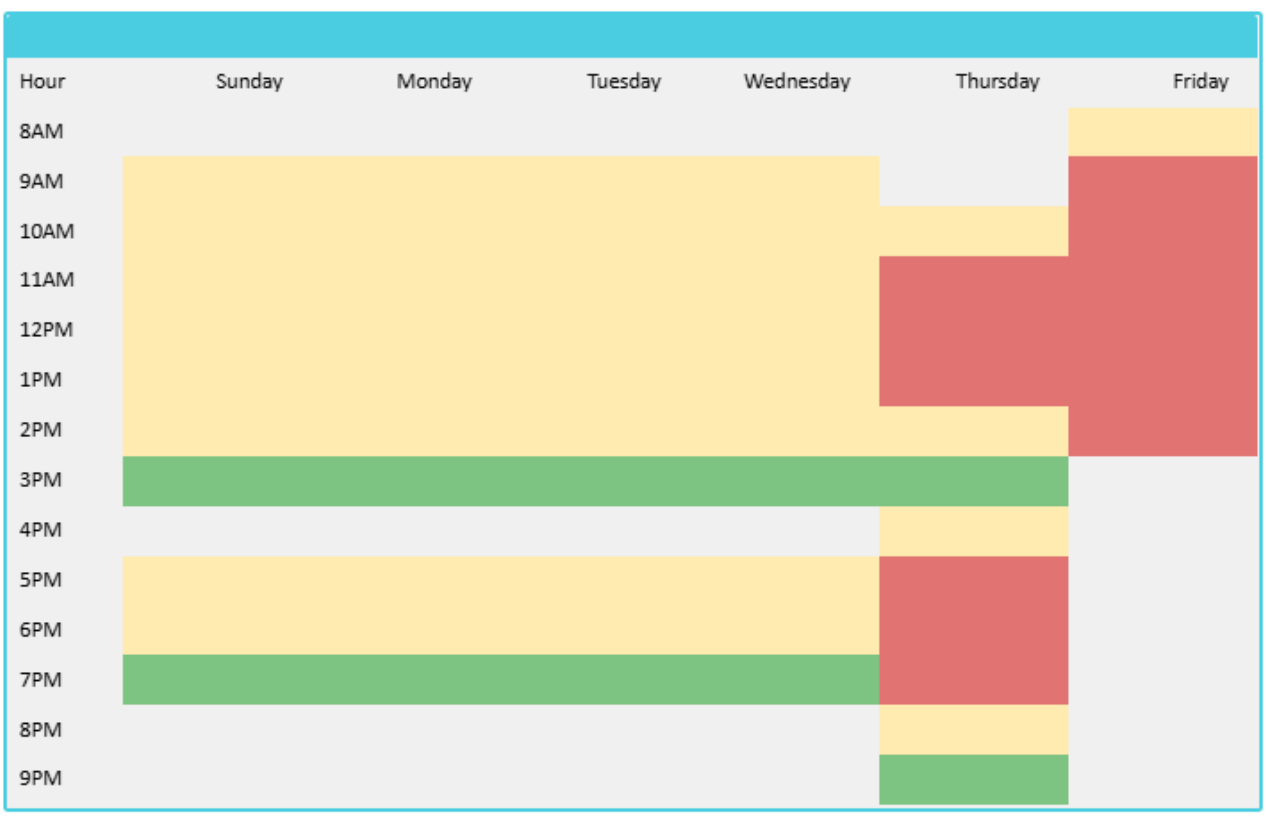
- Peak hour (≥ P75): High customer density *and* strong checkout activity → maximum operational load.
- Dead time (≤ P25): Low traffic and low transactions → minimal operational pressure.
- Regular hour: Balanced, manageable flow.

```
with geo_hourly as (
    -- step 1: calculate unique customers per day and hour based on geolocation data.
    -- This represents physical presence inside the store.
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct device_id) as customers
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role in ('repeat_customer', 'one_time_customer', 'not_paying')
      and accuracy_m <= 30
      and area != 'PARKING'
    group by dow, hour_of_day, dt
),
sales_hourly as (
    -- step 2: calculate checkout activity per day and hour.
    -- This represents real operational pressure at the checkout counters.
    select
        extract(dayofweek from timestamp) as dow,
```

```
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct sale_id) as transactions,
        sum(total) as revenue
    from `bqproj-435911.Final_Project_2025.log_sales`
    group by dow, hour_of_day, dt
),
geo_avg as (
    -- step 3a: compute historical average customer presence by day of week and hour of
day.
    select
        dow,
        hour_of_day,
        avg(customers) as avg_customers
    from geo_hourly
    group by dow, hour_of_day
),
sales_avg as (
    -- step 3b: compute historical average checkout activity by day of week and hour of
day.
    select
        dow,
        hour_of_day,
        avg(transactions) as avg_transactions,
        avg(revenue) as avg_revenue
    from sales_hourly
    group by dow, hour_of_day
),
combined_metrics as (
    -- step 4: combine customer presence and checkout metrics into a single table
aligned by day of week and hour.
    select
        g.dow,
        g.hour_of_day,
        g.avg_customers,
        s.avg_transactions,
        s.avg_revenue
    from geo_avg g
    left join sales_avg s
        on g.dow = s.dow
       and g.hour_of_day = s.hour_of_day
),
normalized_metrics as (
    -- step 5: percentile-based normalization using empirical distribution.
    -- rationale:
    -- - more robust to outliers than min-max scaling
    -- - preserves relative position of each hour in the historical distribution
    -- - values range from 0 to 1, representing percentile rank
    select
        *,
        percent_rank() over (
            order by avg_customers
        ) as customers_norm,
        percent_rank() over (
            order by avg_transactions
        ) as transactions_norm
    from combined_metrics
),
activity_index as (
    -- step 6: composite store activity index based on percentile-normalized metrics.
```

```sql
    -- customer presence receives higher weight because it reflects in-store congestion
and operational load.
    select
        *,
        0.6 * customers_norm
      + 0.4 * transactions_norm as store_activity_index
    from normalized_metrics
),
global_thresholds as (
    -- step 7: calculate global percentiles across all hours.
    -- percentiles are computed on the composite activity index, not per hour, to ensure
consistent classification.
    select
        approx_quantiles(store_activity_index, 100)[safe_offset(25)] as p25_global,
        approx_quantiles(store_activity_index, 100)[safe_offset(75)] as p75_global
    from activity_index
)
-- step 8: final classification of each hour into peak, regular, or dead time
select
    a.dow,
    a.hour_of_day,
    round(a.avg_customers, 1) as avg_customers,
    round(a.avg_transactions, 1) as avg_transactions,
    round(a.avg_revenue, 2) as avg_revenue,
    round(a.store_activity_index, 3) as activity_index,
    case
        when a.store_activity_index >= t.p75_global then 'peak_hour'
        when a.store_activity_index <= t.p25_global then 'dead_time'
        else 'regular_hour'
    end as activity_class
from activity_index a
cross join global_thresholds t
order by a.dow, a.hour_of_day;
```

Weekly Peak & Off-Peak Analysis: Footfall vs. Transaction Trends



This heatmap visualizes a Composite Store Activity Index, which balances physical customer presence (60% weight) with real-time transaction volume (40% weight). By normalizing these metrics into percentiles, the chart identifies specific "operational states" regardless of raw numbers. It allows management to see at a glance when the store is under the most pressure versus when it is under-utilized.

Conclusion of Results
Peak Operational Windows (Red): The store hits its highest activity thresholds (Top 25%) predominantly on Thursday and Friday afternoons. These are critical periods where maximum staffing at checkouts and floor support is required to prevent bottlenecks.

Strategic Dead Hours (Green): Low-utilization "Dead Hours" (Bottom 25%) are identified consistently around 3 PM and 7 PM across the week. These windows represent the best times for non-customer-facing tasks, such as heavy shelf restocking, deep cleaning, or staff shift rotations.

Regular Activity Patterns (Yellow): Most morning hours and the earlier part of the week show a Regular state. This indicates a stable, predictable flow where standard operational procedures are sufficient.

46. Customers-Only Version (Sanity Check)

Objective: Validate whether customer presence alone produces a pattern consistent with the combined model, and identify divergences that reveal behavioral insights.

Statistical / Mathematical Approach

- Uses the same percentile-based normalization (PERCENT_RANK), but applied only to avg_customers.
- Classification thresholds are derived from the customer distribution itself, ensuring methodological consistency with the combined model.

This is intentionally a simpler model, used for validation rather than decision-making.

Retail / Business Rationale
Customer presence is the earliest signal of congestion, but does not guarantee checkout pressure.
This model answers:
"When is the store physically crowded, regardless of purchasing behavior?"

How to Interpret Divergences

- Peak in customers-only, regular in combined
  - High foot traffic, but relatively low conversion.
  - Typical of browsing periods, price checks, or short visits.
  - Operational implication: congestion without proportional revenue.
- Peak in combined, regular in customers-only
  - Fewer customers, but high transaction intensity.
  - Indicates large baskets or fast checkout turnover.
  - Operational implication: fewer people, but high cashier workload.
- Aligned peak in both
  - True high-stress retail periods.
  - Priority windows for staffing and queue management.

This sanity check confirms that the combined index adds explanatory power, rather than duplicating customer counts.

```
with geo_hourly as (
    -- base: daily unique customer presence per day and hour.
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
```

```sql
        date(timestamp) as dt,
        count(distinct device_id) as customers
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role in ('repeat_customer', 'one_time_customer', 'not_paying')
      and accuracy_m <= 30
      and area != 'PARKING'
    group by dow, hour_of_day, dt
),
geo_avg as (
    -- historical average customer presence by day of week and hour of day.
    select
        dow,
        hour_of_day,
        avg(customers) as avg_customers
    from geo_hourly
    group by dow, hour_of_day
),
geo_normalized as (
    -- percentile-based normalization of customer presence only.
    select
        *,
        percent_rank() over (
            order by avg_customers
        ) as customers_norm
    from geo_avg
),
geo_thresholds as (
    -- global thresholds based on customer presence only.
    select
        approx_quantiles(customers_norm, 100)[safe_offset(25)] as p25,
        approx_quantiles(customers_norm, 100)[safe_offset(75)] as p75
    from geo_normalized
)
select
    g.dow,
    g.hour_of_day,
    round(g.avg_customers, 1) as avg_customers,
    round(g.customers_norm, 3) as customers_norm,
    case
        when g.customers_norm >= t.p75 then 'peak_hour'
        when g.customers_norm <= t.p25 then 'dead_time'
        else 'regular_hour'
    end as activity_class_customers_only
from geo_normalized g
cross join geo_thresholds t
order by g.dow, g.hour_of_day;
```

47. Pre-Holiday vs Regular Day Segmentation

Objective

Assess how hourly demand patterns differ on days preceding holidays, isolating structurally abnormal behavior to prevent distortion of the regular operating baseline.

Statistical / Mathematical Approach

Observations are explicitly segmented into two demand regimes:

- pre_holiday (D-1 of official holidays)
- regular_day

Hourly customer presence is aggregated within each regime, and percentile-based thresholds (P25 / P75) are computed separately per segment.

This regime-aware approach prevents mixture bias by ensuring that pre-holiday surges do not inflate regular-day benchmarks.

Retail / Business Rationale

In grocery retail, pre-holiday days exhibit structurally different demand dynamics, including stock-up behavior, larger baskets, and earlier, prolonged peaks.
Treating these days as regular operations would overestimate baseline demand, understate true peak intensity, and lead to systematic understaffing on normal days.

Interpretation of Results

The comparative analysis reveals that Pre-Holiday demand is not just "higher volume," but a structurally different operational regime:

- Volume Surge: Pre-Holiday traffic peaks are ~81% higher than Regular Days, reaching ~200 distinct devices vs. ~110 on a baseline day.
- Temporal Shift (Front-Loading): Peak activity accelerates. Pre-Holiday demand surges at 9 AM and peaks at 10 AM–11 AM, whereas Regular Days follow a traditional 12 PM–1 PM peak.
- Elimination of Lulls: "Dead Hours" (P25) virtually disappear on Pre-Holidays between 9 AM and 7 PM. The store operates in a sustained state of high pressure, leaving no window for midday restocking.
- Threshold Divergence: A "Regular Hour" in a Pre-Holiday regime often carries more physical congestion than a "Peak Hour" in the Regular Day regime, justifying the use of independent, regime-aware benchmarks.

```sql
with holidays as (
    -- official holiday calendar.
    -- serves as the authoritative source for demand regime segmentation.
    select date '2025-06-02' as holiday_date union all  -- shavuot
    select date '2025-09-23' union all                  -- rosh hashana
    select date '2025-09-24' union all                  -- rosh hashana
    select date '2025-10-02' union all                  -- yom kippur
    select date '2025-10-07' union all                  -- sukkot
    select date '2025-10-14'                            -- simchat torah
),
pre_holidays as (
    -- dynamically derive pre-holiday days (D-1).
    -- pre-holiday behavior in supermarkets is structurally different due to stock-up
shopping and increased checkout pressure.
    select
        date_sub(holiday_date, interval 1 day) as dt
    from holidays
),
geo_hourly as (
    -- step 1: calculate hourly customer presence per day.
    -- geolocation-based presence captures physical congestion and store traffic
independently of checkout conversion.
    select
        date(timestamp) as dt,
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        count(distinct device_id) as customers
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role in ('repeat_customer', 'one_time_customer', 'not_paying')
      and accuracy_m <= 30
      and area != 'PARKING'
    group by dt, dow, hour_of_day
),
```

```sql
geo_labeled as (
    -- step 2: label each observation according to demand regime.
    -- pre-holiday days are isolated to prevent their abnormal demand patterns from
contaminating regular-day baselines.
    select
        g.*,
        case
            when p.dt is not null then 'pre_holiday'
            else 'regular_day'
        end as day_type
    from geo_hourly g
    left join pre_holidays p
        on g.dt = p.dt
),
hourly_aggregates as (
    -- step 3: compute historical average customer presence by day type, weekday, and
hour.
    --  this stabilizes noise and reveals structural behavioral patterns within each
regime.
        select
        day_type,
        dow,
        hour_of_day,
        avg(customers) as avg_customers
    from geo_labeled
    group by day_type, dow, hour_of_day
),
thresholds as (
    -- step 4: compute percentile-based thresholds separately for each demand regime.
    -- this ensures that peak and dead hour definitions are regime-aware and
operationally realistic.
    select
        day_type,
        approx_quantiles(avg_customers, 100)[safe_offset(25)] as p25_customers,
        approx_quantiles(avg_customers, 100)[safe_offset(75)] as p75_customers
    from hourly_aggregates
    group by day_type
)
    -- step 5:
    -- classify hours into peak, regular, or dead time
select
    h.day_type,
    h.dow,
    h.hour_of_day,
    round(h.avg_customers, 1) as avg_customers,
    round(t.p25_customers, 1) as p25_customers,
    round(t.p75_customers, 1) as p75_customers,
    case
        when h.avg_customers >= t.p75_customers then 'peak_hour'
        when h.avg_customers <= t.p25_customers then 'dead_hour'
        else 'regular_hour'
    end as activity_class
from hourly_aggregates h
join thresholds t
    on h.day_type = t.day_type
order by h.day_type, h.dow, h.hour_of_day;
```

Analytical Summary

Across the three queries, the analysis establishes a layered, defensible methodology:

1. Combined percentile-based index
   → Primary decision-making model for peak vs dead hours.
2. Customers-only sanity check
   → Behavioral validation and diagnostic tool.
3. Pre-holiday segmentation
   → Context-aware correction for exceptional demand periods.

Together, these approaches:

- Avoid arbitrary thresholds,
- Reflect real supermarket operations,
- Provide explainable, auditable logic suitable for executive review.

b) Operational Optimization

48. Hourly Checkout Demand

Objective: Measure the hourly arrival rate of customers at checkout areas.

Method
Average of distinct customers detected at cash registers per hour and weekday, using geolocation data.

Retail Rationale
In supermarkets, checkout congestion is driven by *arrival rate at registers*, not by total in-store foot traffic.

How to Read the Output
Higher values indicate hours with greater service demand, requiring more open registers or self-checkout support.

```sql
-- hourly checkout demand based on geolocation at cash registers
with checkout_customers as (
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct device_id) as customers_at_checkout
    from `bqproj-435911.Final_Project_2025.geolocation`
    where area = 'CASH_REGISTERS'
      and role in ('repeat_customer', 'one_time_customer')
      and accuracy_m <= 30
    group by dow, hour_of_day, dt
)
select
    dow,
    hour_of_day,
    round(avg(customers_at_checkout), 1) as avg_customers_at_checkout
from checkout_customers
group by dow, hour_of_day
order by dow, hour_of_day;
```

49. Observed Cashier Capacity (Throughput)

Objective: Estimate the real, observed service capacity of a single cashier per hour.

Method
Ratio between average hourly transactions and average active cashiers (transactions per cashier-hour).

Retail Rationale
Using observed productivity avoids theoretical assumptions and reflects real-world constraints such as payment mix, basket size, and operational friction.

How to Read the Output
The result represents a realistic throughput benchmark, used to translate customer demand into required staffing levels.

```sql
-- cashier productivity: transactions handled per cashier per hour
with cashier_presence as (
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct device_id) as cashiers_active
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role = 'cashier'
      and area = 'CASH_REGISTERS'
    group by dow, hour_of_day, dt
),

sales_hourly as (
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct sale_id) as transactions
    from `bqproj-435911.Final_Project_2025.log_sales`
    group by dow, hour_of_day, dt
)

select
    s.dow,
    s.hour_of_day,
    round(avg(s.transactions), 2) as avg_transactions,
    round(avg(c.cashiers_active), 2) as avg_cashiers,
    safe_divide(
        avg(s.transactions),
        avg(c.cashiers_active)
    ) as transactions_per_cashier_hour
from sales_hourly s
join cashier_presence c
  on s.dow = c.dow
 and s.hour_of_day = c.hour_of_day
 and s.dt = c.dt
group by s.dow, s.hour_of_day
order by s.dow, s.hour_of_day;
```

50. Dwell Time Validation

Objective: Validate whether the assumed checkout service capacity is sustainable from the customer experience perspective.

Method
Average dwell time (in minutes) by weekday and hour, using sales data.

Retail Rationale
In supermarkets, excessive dwell time is a strong proxy for checkout congestion and queue buildup, especially during peak hours.

How to Read the Output
If dwell time increases while cashier capacity remains stable, it indicates service bottlenecks and validates the need for more capacity or automation.

```
-- average dwell time to validate service capacity assumptions
select
    extract(dayofweek from timestamp) as dow,
    extract(hour from timestamp) as hour_of_day,
    avg(dwell_minutes) as avg_dwell_minutes
from `bqproj-435911.Final_Project_2025.log_sales`
group by dow, hour_of_day
order by dow, hour_of_day;
```

51. Recommended number of cashiers per hour

Objective: Translate hourly checkout demand into an optimal number of cashiers per shift.

Method
Divide average hourly checkout demand by observed cashier throughput, rounding up to ensure service coverage.

Retail Rationale
Staffing decisions in retail must be demand-driven and based on observed productivity, not fixed staffing rules.

How to Read the Output
The result represents the minimum number of cashiers required per hour to meet demand without excessive waiting times.

```
-- recommended number of cashiers per hour
with demand as (
    select
        dow,
        hour_of_day,
        avg(customers_at_checkout) as demand_customers
    from (
        select
            extract(dayofweek from timestamp) as dow,
            extract(hour from timestamp) as hour_of_day,
            date(timestamp) as dt,
            count(distinct device_id) as customers_at_checkout
        from `bqproj-435911.Final_Project_2025.geolocation`
        where area = 'CASH_REGISTERS'
          and role in ('repeat_customer', 'one_time_customer')
          and accuracy_m <= 30
        group by dow, hour_of_day, dt
    )
    group by dow, hour_of_day
),
capacity as (
    select
        dow,
        hour_of_day,
        avg(transactions_per_cashier_hour) as capacity_per_cashier
    from (
        select
            s.dow,
            s.hour_of_day,
            safe_divide(
                avg(s.transactions),
                avg(c.cashiers_active)
            ) as transactions_per_cashier_hour
```

```
        from (
            select
                extract(dayofweek from timestamp) as dow,
                extract(hour from timestamp) as hour_of_day,
                date(timestamp) as dt,
                count(distinct sale_id) as transactions
            from `bqproj-435911.Final_Project_2025.log_sales`
            group by dow, hour_of_day, dt
        ) s
        join (
            select
                extract(dayofweek from timestamp) as dow,
                extract(hour from timestamp) as hour_of_day,
                date(timestamp) as dt,
                count(distinct device_id) as cashiers_active
            from `bqproj-435911.Final_Project_2025.geolocation`
            where role = 'cashier'
              and area = 'CASH_REGISTERS'
            group by dow, hour_of_day, dt
        ) c
        on s.dow = c.dow
       and s.hour_of_day = c.hour_of_day
       and s.dt = c.dt
        group by s.dow, s.hour_of_day
    )
    group by dow, hour_of_day
)
select
    d.dow,
    d.hour_of_day,
    round(d.demand_customers, 1) as avg_customers_at_checkout,
    round(c.capacity_per_cashier, 1) as customers_per_cashier_hour,
    ceil(
        safe_divide(d.demand_customers, c.capacity_per_cashier)
    ) as recommended_cashiers
from demand d
join capacity c
  on d.dow = c.dow
 and d.hour_of_day = c.hour_of_day
order by d.dow, d.hour_of_day;
```

52. Self-checkout investment windows

Objective: Identify hours with structurally low checkout demand that are suitable for self-checkout-led operation.

Method

Classify hours below the 25th percentile of average checkout demand as high self-checkout potential.

Retail Rationale

In supermarkets, self-checkout is most efficient during low-volume periods, where fixed cashier labor costs outweigh service complexity.

How to Read the Output

Hours flagged as *high_self_checkout_potential* are candidates to reduce staffed lanes and rely more on self-checkout without harming service levels.

```
-- identify hours suitable for self-checkout investment
select
    dow,
    hour_of_day,
```

```
    avg_customers_at_checkout,
    case
        when avg_customers_at_checkout <= percentile_cont(avg_customers_at_checkout,
0.25)
             over () then 'high_self_checkout_potential'
        else 'standard_staffing'
    end as recommendation
from (
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        count(distinct device_id) as avg_customers_at_checkout
    from `bqproj-435911.Final_Project_2025.geolocation`
    where area = 'CASH_REGISTERS'
      and role in ('repeat_customer', 'one_time_customer')
      and accuracy_m <= 30
    group by dow, hour_of_day
)
order by dow, hour_of_day;
```

53. Recommended cashiers vs. observed staffing

Objective: Compare required cashier capacity against actual staffing levels to identify gaps and optimization opportunities.

Method

Estimate required cashiers by dividing average checkout demand by observed cashier productivity (transactions per cashier hour).

Retail Rationale

Retail staffing must balance labor cost with queue risk; this approach grounds decisions in real operational throughput.

How to Read the Output

Differences between recommended and observed cashiers highlight overstaffing (cost leakage) or understaffing (service risk).

```
-- recommended number of cashiers per hour with observed staffing comparison
with checkout_demand as (
    -- customer demand at checkout per hour
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct device_id) as customers_at_checkout
    from `bqproj-435911.Final_Project_2025.geolocation`
    where area = 'CASH_REGISTERS'
      and role in ('repeat_customer', 'one_time_customer', 'not_paying')
      and accuracy_m <= 30
    group by dow, hour_of_day, dt
),
cashier_presence as (
    -- actual number of cashiers active per hour
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct device_id) as active_cashiers
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role = 'cashier'
      and area = 'CASH_REGISTERS'
```

```sql
    group by dow, hour_of_day, dt
),
sales_hourly as (
    -- checkout throughput
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct sale_id) as transactions
    from `bqproj-435911.Final_Project_2025.log_sales`
    group by dow, hour_of_day, dt
),
hourly_metrics as (
    select
        d.dow,
        d.hour_of_day,
        avg(d.customers_at_checkout) as avg_customers_at_checkout,
        avg(c.active_cashiers) as avg_active_cashiers,
        avg(s.transactions) as avg_transactions,
        safe_divide(
            avg(s.transactions),
            avg(c.active_cashiers)
        ) as transactions_per_cashier_hour
    from checkout_demand d
    join cashier_presence c
      on d.dow = c.dow
     and d.hour_of_day = c.hour_of_day
     and d.dt = c.dt
    join sales_hourly s
      on d.dow = s.dow
     and d.hour_of_day = s.hour_of_day
     and d.dt = s.dt
    group by d.dow, d.hour_of_day
)
select
    dow,
    hour_of_day,
    round(avg_customers_at_checkout, 1) as avg_customers_at_checkout,
    round(avg_active_cashiers, 1) as avg_active_cashiers,
    round(transactions_per_cashier_hour, 1) as customers_per_cashier_hour,
    ceil(
        safe_divide(avg_customers_at_checkout, transactions_per_cashier_hour)
    ) as recommended_cashiers
from hourly_metrics
order by dow, hour_of_day;
```

54. Hourly overstaffing / understaffing detection (Weekday × Hour)

Objective: Identify hours with staffing imbalance (overstaffed vs. understaffed) by comparing observed cashier presence against demand-driven requirements.

Statistical / Operational Approach
Average checkout demand is divided by observed cashier productivity (transactions per cashier per hour) to estimate the required number of cashiers. The gap between required and observed staffing defines the staffing status.

Key Metrics / KPIs

- avg_customers_at_checkout
- transactions_per_cashier_hour
- recommended_cashiers

- staffing_gap
- staffing_status

Retail Rationale
Supermarkets incur high fixed labor costs at checkout. Detecting systematic overstaffing reveals cost-saving opportunities, while understaffing highlights service and queue-risk periods.

Interpretation of Results

- *Understaffed*: risk of queues, longer dwell time, and lost sales
- *Overstaffed*: labor inefficiency during structurally low-demand hours

```
-- recommended number of cashiers per hour with staffing gap analysis
with checkout_demand as (
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct device_id) as customers_at_checkout
    from `bqproj-435911.Final_Project_2025.geolocation`
    where area = 'CASH_REGISTERS'
      and role in ('repeat_customer', 'one_time_customer', 'not_paying')
      and accuracy_m <= 30
    group by dow, hour_of_day, dt
),
cashier_presence as (
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct device_id) as active_cashiers
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role = 'cashier'
      and area = 'CASH_REGISTERS'
    group by dow, hour_of_day, dt
),
sales_hourly as (
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct sale_id) as transactions
    from `bqproj-435911.Final_Project_2025.log_sales`
    group by dow, hour_of_day, dt
),
hourly_metrics as (
    select
        d.dow,
        d.hour_of_day,
        avg(d.customers_at_checkout) as avg_customers_at_checkout,
        avg(c.active_cashiers) as avg_active_cashiers,
        avg(s.transactions) as avg_transactions,
        safe_divide(
            avg(s.transactions),
            nullif(avg(c.active_cashiers), 0)
        ) as transactions_per_cashier_hour
    from checkout_demand d
    join cashier_presence c
      on d.dow = c.dow
     and d.hour_of_day = c.hour_of_day
```

```
     and d.dt = c.dt
    join sales_hourly s
      on d.dow = s.dow
     and d.hour_of_day = s.hour_of_day
     and d.dt = s.dt
    group by d.dow, d.hour_of_day
)
select
    dow,
    hour_of_day,
    round(avg_customers_at_checkout, 1) as avg_customers_at_checkout,
    round(avg_active_cashiers, 1) as avg_active_cashiers,
    round(transactions_per_cashier_hour, 1) as customers_per_cashier_hour,

    ceil(
        safe_divide(avg_customers_at_checkout, transactions_per_cashier_hour)
    ) as recommended_cashiers,

    round(
        avg_active_cashiers
      - ceil(
            safe_divide(avg_customers_at_checkout, transactions_per_cashier_hour)
        ),
        1
    ) as staffing_gap,

    case
        when avg_active_cashiers <
             ceil(safe_divide(avg_customers_at_checkout, transactions_per_cashier_hour))
            then 'understaffed'
        when avg_active_cashiers >
             ceil(safe_divide(avg_customers_at_checkout, transactions_per_cashier_hour))
            then 'overstaffed'
        else 'adequately_staffed'
    end as staffing_status
from hourly_metrics
order by dow, hour_of_day;
```

55. Staffing analysis with peak / dead hour classification (p75 Demand)

Objective: Combine staffing adequacy and demand intensity to classify each hour as peak, normal, or dead, using a high-demand (p75) stress scenario.

Statistical / Operational Approach

Uses the 75th percentile of checkout demand to model peak pressure, derives a pressure index, and classifies hours via percentile-based thresholds per weekday.

Key Metrics / KPIs

- p75_customers
- recommended_cashiers_p75
- pressure_index
- staffing_status
- demand_classification

Retail Rationale

Retail operations must be resilient to demand spikes, not just averages. Using p75 demand ensures staffing recommendations remain robust during busy but recurring peak conditions.

```
with checkout_demand as (
    select
        extract(dayofweek from timestamp) as dow,
```

```
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct device_id) as customers_at_checkout
    from `bqproj-435911.Final_Project_2025.geolocation`
    where area = 'CASH_REGISTERS'
      and role in ('repeat_customer', 'one_time_customer')
      and accuracy_m <= 30
    group by dow, hour_of_day, dt
),
cashier_presence as (
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct device_id) as active_cashiers
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role = 'cashier'
      and area = 'CASH_REGISTERS'
    group by dow, hour_of_day, dt
),
sales_hourly as (
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct sale_id) as transactions
    from `bqproj-435911.Final_Project_2025.log_sales`
    group by dow, hour_of_day, dt
),
hourly_distribution as (
    select
        d.dow,
        d.hour_of_day,
        approx_quantiles(d.customers_at_checkout, 100)[offset(75)] as p75_customers,
        avg(c.active_cashiers) as avg_active_cashiers,
        safe_divide(
            avg(s.transactions),
            avg(c.active_cashiers)
        ) as transactions_per_cashier_hour
    from checkout_demand d
    join cashier_presence c
      on d.dow = c.dow
     and d.hour_of_day = c.hour_of_day
     and d.dt = c.dt
    join sales_hourly s
      on d.dow = s.dow
     and d.hour_of_day = s.hour_of_day
     and d.dt = s.dt
    group by d.dow, d.hour_of_day
),
staffing_analysis as (
    select
        dow,
        hour_of_day,
        p75_customers,
        avg_active_cashiers,
        transactions_per_cashier_hour,
        ceil(
            safe_divide(
                p75_customers,
                nullif(transactions_per_cashier_hour, 0)
```

```sql
            )
        ) as recommended_cashiers_p75,
        safe_divide(
            p75_customers,
            nullif(transactions_per_cashier_hour, 0)
        ) as pressure_index
    from hourly_distribution
),
pressure_thresholds as (
    select
        dow,
        approx_quantiles(pressure_index, 100)[offset(75)] as p75_pressure,
        approx_quantiles(pressure_index, 100)[offset(25)] as p25_pressure
    from staffing_analysis
    group by dow
)
select
    s.dow,
    s.hour_of_day,
    s.p75_customers,
    round(s.avg_active_cashiers, 1) as avg_active_cashiers,
    s.recommended_cashiers_p75,
    round(s.avg_active_cashiers - s.recommended_cashiers_p75, 1) as staff_gap,
    case
        when s.avg_active_cashiers > s.recommended_cashiers_p75 then 'overstaffed'
        when s.avg_active_cashiers < s.recommended_cashiers_p75 then 'understaffed'
        else 'well_staffed'
    end as staffing_status,
    case
        when s.pressure_index >= t.p75_pressure then 'peak_hour'
        when s.pressure_index <= t.p25_pressure then 'dead_hour'
        else 'normal_hour'
    end as demand_classification
from staffing_analysis s
join pressure_thresholds t
  on s.dow = t.dow
order by s.dow, s.hour_of_day;
```

Interpretation of Results

- *Peak_hour*: requires operational reinforcement (staffing or self-checkout support)
- *Dead_hour*: opportunity for cost optimization and automation
- *Normal_hour*: balanced operation

Staffing analysis with peak / dead hour classification

HOUR	Sunday	Monday	Tuesday	Wednesday	Thursday	Friday
8AM	-	-	-	-	-	
9AM					-	
10AM						
11AM						
12PM						
1PM						
2PM						
3PM						-
4PM	-	-	-	-		-
5PM						-
6PM						-
7PM						-
8PM	-	-	-	-		-

56. Self-checkout potential based on low operational utilization

Objective: Identify hours where traditional staffed checkouts are underutilized and self-checkout can be prioritized without service risk.

Analytical Logic
Calculates average customers per active cashier by weekday and hour. Hours below the 25th percentile of utilization are flagged as high self-checkout potential.

Key Metrics / KPIs

- avg_customers
- avg_cashiers
- customers_per_cashier
- checkout_strategy

Business Rationale
Low utilization periods represent inefficient labor allocation. Shifting demand to self-checkout during these hours reduces costs while maintaining service availability.

Executive Interpretation

- *High self-checkout potential*: reduce staffed lanes, prioritize automation
- *Standard cashiers preferred*: maintain human checkout presence

```
-- identify hours with high self-checkout potential based on low operational utilization
with hourly_operations as (
    select
        extract(dayofweek from g.timestamp) as dow,
        extract(hour from g.timestamp) as hour_of_day,
        date(g.timestamp) as dt,
        count(distinct g.device_id) as customers_at_checkout,
        count(distinct c.device_id) as active_cashiers
    from `bqproj-435911.Final_Project_2025.geolocation` g
    left join `bqproj-435911.Final_Project_2025.geolocation` c
```

```
      on date(g.timestamp) = date(c.timestamp)
     and extract(hour from g.timestamp) = extract(hour from c.timestamp)
     and extract(dayofweek from g.timestamp) = extract(dayofweek from c.timestamp)
     and c.role = 'cashier'
     and c.area = 'CASH_REGISTERS'
    where g.area = 'CASH_REGISTERS'
      and g.role in ('repeat_customer', 'one_time_customer', 'not_paying')
      and g.accuracy_m <= 30
    group by dow, hour_of_day, dt
),
hourly_avg as (
    select
        dow,
        hour_of_day,
        avg(customers_at_checkout) as avg_customers,
        avg(active_cashiers) as avg_cashiers,
        safe_divide(
            avg(customers_at_checkout),
            nullif(avg(active_cashiers), 0)
        ) as customers_per_cashier
    from hourly_operations
    group by dow, hour_of_day
),
thresholds as (
    select
        approx_quantiles(customers_per_cashier, 100)[safe_offset(25)] as p25_utilization
    from hourly_avg
)
select
    h.dow,
    h.hour_of_day,
    round(h.avg_customers, 1) as avg_customers,
    round(h.avg_cashiers, 1) as avg_cashiers,
    round(h.customers_per_cashier, 2) as customers_per_cashier,
    case
        when h.customers_per_cashier <= t.p25_utilization
            then 'high_self_checkout_potential'
        else 'standard_cashiers_preferred'
    end as checkout_strategy
from hourly_avg h
cross join thresholds t
order by h.dow, h.hour_of_day;
```

57. Integrated staffing × demand × self-checkout decision KPI

Objective: Provide a unified, decision-ready KPI that links demand intensity, staffing adequacy, and the strategic role of self-checkout.

Analytical Framework

Uses p75 demand to stress-test staffing needs, classifies hours by demand pressure (peak / regular / dead), and maps each scenario to a self-checkout recommendation with managerial rationale.

Key Metrics / KPIs

- demand_classification
- staffing_status
- staff_gap
- self_checkout_role
- self_checkout_rationale

Business Rationale
Self-checkout should not be deployed uniformly. Its role varies by operational context: cost optimization in low demand, capacity buffer in normal hours, and support mechanism during peaks.

Executive Interpretation

- *Peak + understaffed*: self-checkout as operational support
- *Dead time*: self-checkout as cost-reduction lever
- *Regular hours*: flexible capacity management

```sql
-- Staffing × Demand × Self-Checkout Decision KPI
with checkout_demand as (
    /*
    customer demand at checkout per hour (daily granularity)
    */
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct device_id) as customers_at_checkout
    from `bqproj-435911.Final_Project_2025.geolocation`
    where area = 'CASH_REGISTERS'
      and role in ('repeat_customer', 'one_time_customer', 'not_paying')
      and accuracy_m <= 30
    group by dow, hour_of_day, dt
),
cashier_presence as (
    /*
    actual number of active cashiers per hour
    */
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct device_id) as active_cashiers
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role = 'cashier'
      and area = 'CASH_REGISTERS'
    group by dow, hour_of_day, dt
),
sales_hourly as (
    /*
    real checkout throughput per hour
    */
    select
        extract(dayofweek from timestamp) as dow,
        extract(hour from timestamp) as hour_of_day,
        date(timestamp) as dt,
        count(distinct sale_id) as transactions
    from `bqproj-435911.Final_Project_2025.log_sales`
    group by dow, hour_of_day, dt
),
hourly_distribution as (
    /*
    build demand distribution and cashier capacity baseline
    */
    select
        d.dow,
        d.hour_of_day,
        approx_quantiles(d.customers_at_checkout, 100)[offset(75)] as p75_customers,
        avg(c.active_cashiers) as avg_active_cashiers,
```

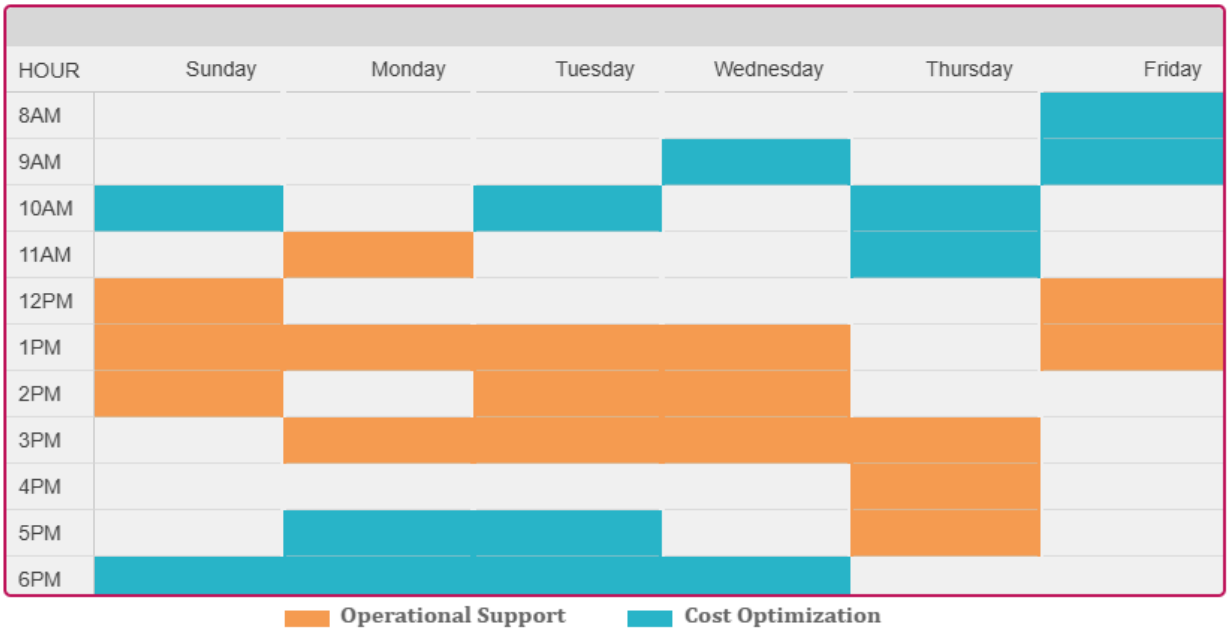
```sql
        safe_divide(
            avg(s.transactions),
            avg(c.active_cashiers)
        ) as transactions_per_cashier_hour
    from checkout_demand d
    join cashier_presence c
      on d.dow = c.dow
     and d.hour_of_day = c.hour_of_day
     and d.dt = c.dt
    join sales_hourly s
      on d.dow = s.dow
     and d.hour_of_day = s.hour_of_day
     and d.dt = s.dt
    group by d.dow, d.hour_of_day
),
staffing_analysis as (
    /*
    estimate required staffing under high-demand conditions (p75)
    */
    select
        dow,
        hour_of_day,
        p75_customers,
        avg_active_cashiers,
        transactions_per_cashier_hour,
        ceil(
            safe_divide(
                p75_customers,
                nullif(transactions_per_cashier_hour, 0)
            )
        ) as recommended_cashiers_p75,
        safe_divide(
            p75_customers,
            nullif(transactions_per_cashier_hour, 0)
        ) as pressure_index
    from hourly_distribution
),
pressure_thresholds as (
    /*
    demand thresholds per day of week
    */
    select
        dow,
        approx_quantiles(pressure_index, 100)[offset(75)] as p75_pressure,
        approx_quantiles(pressure_index, 100)[offset(25)] as p25_pressure
    from staffing_analysis
    group by dow
),
staffing_and_demand_classification as (
    /*
    combine staffing gap and demand intensity
    */
    select
        s.dow,
        s.hour_of_day,
        s.p75_customers,
        round(s.avg_active_cashiers, 1) as avg_active_cashiers,
        s.recommended_cashiers_p75,
        round(s.avg_active_cashiers - s.recommended_cashiers_p75, 1) as staff_gap,
        case
            when s.avg_active_cashiers > s.recommended_cashiers_p75 then 'overstaffed'
```

```sql
            when s.avg_active_cashiers < s.recommended_cashiers_p75 then 'understaffed'
            else 'well_staffed'
        end as staffing_status,
        case
            when s.pressure_index >= t.p75_pressure then 'peak_hour'
            when s.pressure_index <= t.p25_pressure then 'dead_time'
            else 'regular_hour'
        end as demand_classification
    from staffing_analysis s
    join pressure_thresholds t
      on s.dow = t.dow
)
-- final output: automatic self-checkout decision kpi
select
    dow,
    hour_of_day,
    demand_classification,
    staffing_status,
    -- automatic self-checkout decision kpi
    'yes' as self_checkout_recommended,

    -- business rationale for self-checkout usage
    case
        when demand_classification = 'dead_time'
            then 'cost_optimization'
        when demand_classification = 'regular_hour'
            then 'flex_capacity_buffer'
        when demand_classification = 'peak_hour'
            then 'operational_support'
        else 'undefined'
    end as self_checkout_role,

    -- managerial interpretation
    case
        when demand_classification = 'peak_hour'
             and staffing_status = 'understaffed'
            then 'use self-checkout as support, not replacement for staffed registers'
        when demand_classification = 'peak_hour'
             and staffing_status = 'overstaffed'
            then 'use self-checkout to reduce marginal labor cost during predictable
peaks'
        when demand_classification = 'dead_time'
            then 'prioritize self-checkout and reduce staffed registers'
        else 'balanced use of self-checkout'
    end as self_checkout_rationale
from staffing_and_demand_classification
order by dow, hour_of_day;
```

Self-Checkout Operational Decision Matrix



The visualization serves as a dynamic staffing guide. It identifies the critical hours where Self-Checkout acts as a safety net for high demand (🟧Operational Support) and when it serves as the primary service point to reduce labor overhead during low-traffic periods (🟦Cost Optimization).

Interpretation of Results

This query evaluates hourly checkout conditions by jointly analyzing customer demand intensity and staffing adequacy, and translates this interaction into a clear self-checkout usage rationale. Demand classification (peak_hour, regular_hour, dead_time) captures how intense customer pressure is relative to historical norms for each day of the week.
Staffing status (understaffed, well_staffed, overstaffed) compares actual cashier availability against the recommended staffing level required to handle high-demand (p75) conditions.

The combination of these two dimensions determines the operational role of self-checkout:

- *Peak demand + understaffed*: Self-checkout is positioned as operational support, helping absorb excess demand without fully replacing staffed registers.
- *Peak demand + overstaffed*: Self-checkout serves as a cost-optimization lever, reducing marginal labor costs during predictable demand spikes.
- *Dead time (low demand)*: Self-checkout becomes the primary channel, allowing a reduction in staffed registers while maintaining service availability.
- *Regular hours or balanced conditions*: Self-checkout acts as a flexibility buffer, smoothing short-term demand fluctuations without structural staffing changes.

Overall, the self_checkout_rationale column converts quantitative demand-and-staffing signals into actionable, manager-ready guidance on when self-checkout should support capacity, optimize costs, or replace staffed checkout lanes.

<u>Analytical Summary</u>

This SQL script builds an end-to-end operational analysis of checkout performance by integrating customer demand, cashier staffing, and sales throughput at an hourly and weekday level.

The analysis starts by validating real operating hours and identifying periods with customer presence but no sales, highlighting potential unexpected closures or operational inconsistencies. It then measures customer flow at checkout areas, distinguishing between low-,

normal-, and high-demand hours using distribution-based thresholds rather than fixed assumptions.

Subsequent queries estimate cashier productivity through transactions per cashier-hour and translate demand into recommended staffing levels, enabling the detection of systematic understaffing and overstaffing patterns by hour and day of week.
The script further classifies hours into peak, regular, and dead periods using pressure indexes derived from p75 demand, ensuring staffing decisions are resilient to demand variability.
Finally, the analysis evaluates self-checkout suitability, identifying low-utilization windows and defining the strategic role of self-checkout as a cost-optimization tool, capacity buffer, or operational support mechanism, depending on demand intensity and staffing gaps.

Overall, this framework transforms raw geolocation and sales data into actionable workforce and automation insights, supporting data-driven decisions on staffing allocation and self-checkout investment.

Part 4 - Regression Analysis

VII. Unique Device Count Over Time – Regression 1

Objective
The objective of this analysis is to evaluate whether the number of weekly customers at the Yavne supermarket branch shows a systematic growth or decline over time, and to infer branch performance ("supermarket success") based on customer traffic trends.

58. Data Preparation & Business Logic

Customer presence was derived from geolocation data between June 1st and November 30th, 2025, aligned with the project scope.
Key retail-driven decisions:

- A customer is defined as a unique device_id, reflecting real visitor count rather than transactions.
- Parking areas were excluded to avoid transitional noise and false positives.
- Employees were excluded to ensure demand reflects customer behavior only.
- Data was aggregated by Israeli commercial weeks (Sunday–Saturday), ensuring alignment with local retail operations.
- Weekly unique customers were calculated as the number of distinct devices per week, serving as a proxy for weekly footfall.
- The final partial week (week 27) was removed because it contained only a single operational day (Sunday), which would artificially depress weekly demand and bias the regression.

A numeric week index was created to enable linear regression, with week number as the independent variable (X) and weekly unique customers as the dependent variable (Y).

Statistical Model
A simple linear regression was estimated:

$$\text{Unique Customers}_t = \beta_0 + \beta_1 \cdot \text{Week Number}_t + \varepsilon_t$$

Where:

- β_1 (slope) captures the weekly trend in customer volume.
- β_0(intercept) represents the estimated baseline customer level.

- Ordinary Least Squares (OLS) was used, consistent with trend analysis over time.

```sql
with base_visits as (
    /*
    step 1:
    select valid in-store customer visits
    business logic:
    - a customer is defined as a unique device_id
    - customers may visit multiple times, but are deduplicated later
    - parking area is excluded to avoid transitional noise
    - only customers (no employees)
    - restrict analysis period to project scope
    */
    select
        date(timestamp) as visit_date,
        device_id
    from `bqproj-435911.Final_Project_2025.geolocation`
    where role in ('repeat_customer', 'one_time_customer', 'not_paying')
      and accuracy_m <= 30
      and area != 'PARKING'
      and date(timestamp) between date '2025-06-01' and date '2025-11-30'
),
weekly_aggregation as (
    /*
    step 2:
    aggregate customer presence by commercial week (Israel standard)
    technical rationale:
    - the business week starts on sunday
    - date_trunc with week(sunday) ensures correct local alignment
    - distinct device_id per week is used as a proxy for unique customers
    */
    select
        -- explicit commercial week start (sunday)
        date_trunc(visit_date, week(sunday)) as week_start_date,

        -- weekly unique customers
        count(distinct device_id) as unique_devices
    from base_visits
    group by week_start_date
),
weekly_indexed as (
    /*
    step 3:
    create a continuous numeric time index for regression
    modeling rationale:
    - regression requires an ordered numeric independent variable
    - dense_rank guarantees continuity even if weeks are missing
    */
    select
        week_start_date,
        unique_devices,
        dense_rank() over (
            order by week_start_date
        ) as week_number
    from weekly_aggregation
)
-- final regression-ready dataset
select
    week_number,          -- independent variable (x)
    unique_devices        -- dependent variable (y)
from weekly_indexed
```

```
order by week_number;
```

Results Summary

- Slope (β_1): `-0.85`
  Indicates a *slight downward trend* in weekly unique customers over the observed period.
- Intercept (β_0): `~518`
  Represents the estimated average weekly customer baseline.
- R^2: `0.12`
  Time explains approximately `12%` of the variation in weekly customer counts, which is expected in retail environments where demand is influenced by promotions, holidays, seasonality, and external events.
- p-value (two-tailed): `0.083`

Interpretation & Business Implications

- The negative slope suggests a mild decline in foot traffic, not a sharp contraction.
- The model is not statistically significant at the `5%` level, but it is borderline at the `10%` level, which is often acceptable for exploratory business diagnostics.
- The low R^2 indicates that time alone is not the main driver of customer volume, reinforcing the need to incorporate additional variables (e.g., promotions, holidays, staffing levels, operational changes).
- From a retail management perspective, this result does not indicate structural failure, but rather a potential early warning signal that warrants monitoring and complementary analysis.

A two-tailed test was applied because:

- The analysis did not assume a priori whether customer traffic would increase or decrease.
- The business question is directional-agnostic: *any significant change* (growth or decline) is relevant.
- Using a one-tailed test would only be appropriate if management had a strong prior hypothesis about the direction of change, which was not the case here.

Therefore, the two-tailed approach is statistically correct and methodologically conservative.

Conclusion
The regression reveals a slight downward trend in weekly customer traffic at the Yavne branch, with limited explanatory power from time alone. While not strongly statistically significant, the results provide actionable insight: customer volume should be monitored alongside operational and commercial drivers to assess long-term branch performance and support data-driven retail decisions.

VIII. Purchase Amount vs. Dwell Time (Minutes)– Regression 2

Objective : The objective of Regression 2 was to analyze the relationship between time spent in store (dwell_minutes) and purchase value (purchase_amount), in order to understand whether longer in-store duration translates into higher customer spending and how this relationship behaves statistically and behaviorally.

This regression focuses on monetization efficiency of time, complementing earlier analyses related to traffic and visit behavior.

59. Data Preparation and Modeling Decisions

➢ **Data Selection (BigQuery)**

The dataset was prepared in BigQuery using a structured, multi-step approach:

- Only completed sales transactions were considered.
- Dwell time is only observable for app users, therefore all records with NULL dwell time were excluded.
- The analysis was restricted to the project time window (June 1, 2025 – November 30, 2025).

This ensured that the regression was performed only on observable, behaviorally valid sessions.

➢ **Data Cleaning**

Additional safeguards were applied:

- Transactions with zero or negative dwell time were removed (invalid sessions).
- Transactions with zero purchase value were excluded, as they do not represent actual buying behavior.

These filters reduced noise and prevented distortions in slope estimation.

➢ **Derived Behavioral Metric**

A diagnostic KPI, purchase_per_minute, was created:

$$\text{purchase_per_minute} = \text{purchase_amount} / \text{dwell_minutes}$$

Although not used as the dependent variable, this metric supported behavioral interpretation (fast vs. slow shoppers).

➢ **Baseline Linear Regression (Level-Level)**

Model Specification

$$\text{purchase_amount} = \beta 0 + \beta 1 \cdot \text{dwell_minutes}$$

Key Results

- Slope (β_1): 6.46
- Intercept (β_0): 21.74
- R^2: 0.759
- p-value (two-tailed): ≈ 0
- 95% Confidence Interval for β_1: [6.42, 6.50]

```
with valid_sales as (
    /*
    step 1:
    select valid sales records with observable dwell time
    business logic:
    - dwell time (minutes_dwell) is only available for app users
    - customers without app have NULL dwell time and must be excluded
    - each row represents a completed purchase
    - restrict analysis to project time window
    */
    select
        sale_id,
        customer_id,
        subtotal as purchase_amount,
        dwell_minutes
    from `bqproj-435911.Final_Project_2025.log_sales`
    where dwell_minutes is not null
      and date(timestamp) between date '2025-06-01' and date '2025-11-30'
),
cleaned_data as (
    /*
```

```sql
    step 2:
    basic data quality safeguards

    modeling rationale:
    - remove zero or negative dwell times (invalid sessions)
    - remove zero-value purchases (non-representative transactions)
    */
    select
        customer_id,
        purchase_amount,
        dwell_minutes
    from valid_sales
    where dwell_minutes > 0
      and purchase_amount > 0
),
final_dataset as (
    /*
    step 3:
    create additional behavioral metric required by the exercise

    spending_rate:
    - represents spending efficiency per minute in store
    - useful for behavioral interpretation (fast vs slow shoppers)
    - not the dependent variable of the regression, but a diagnostic KPI
    */
    select
        customer_id,
        dwell_minutes,                          -- independent variable (X)
        purchase_amount,                       -- dependent variable (Y)
        safe_divide(purchase_amount, dwell_minutes)
            as purchase_per_minute             -- derived behavioral metric
    from cleaned_data
)
-- final regression-ready output
select
    dwell_minutes,
    purchase_amount,
    purchase_per_minute
from final_dataset
order by dwell_minutes;
```

Interpretation

Statistically, the model shows a very strong and significant relationship between dwell time and purchase amount. On average, each additional minute in store is associated with an increase of approximately 6.46 monetary units in purchase value.

However, diagnostic analysis of the residuals revealed a clear funnel-shaped pattern, indicating heteroscedasticity. Residual variance increases as dwell time increases, meaning that the model is much more precise for short visits and increasingly unreliable for long visits.

This violates a key assumption of OLS regression and weakens inference at higher dwell times.

- ➢ **Log-Linear Regression (Correcting Heteroscedasticity)**

Motivation

To address heteroscedasticity and stabilize variance, the dependent variable was log-transformed:

$$\log(\text{purchase_amount}) = \beta 0 + \beta 1 \cdot \text{dwell_minutes}$$

Results

- Slope (β_1): 0.0187
- Intercept (β_0): 4.736
- R^2: 0.758
- p-value: effectively 0 (given very high t-statistic)
- Residual sanity check: ~0

Interpretation

In a log-linear model, the slope represents a percentage effect. A coefficient of 0.0187 implies that each additional minute in store increases purchase value by approximately 1.87%, on average.
The transformation successfully eliminated heteroscedasticity, producing a much more stable residual distribution.

However, residual diagnostics revealed a systematic curvature ("smile" pattern), indicating that the relationship between time and spending is not fully linear, even in log space. The model tends to underpredict mid-range dwell times and overpredict extreme values.
This suggested the presence of structurally different shopping behaviors across dwell-time ranges.

➢ **Time-Based Segmentation (Business-Driven Approach)**

Rationale

Rather than forcing a single functional form across all customers, a time-based segmentation was introduced to align the statistical model with retail behavior.
Customers were segmented as:

- Quick Trip: ≤ 10 minutes
- Regular Shopping: 11–30 minutes
- Long Stay: > 30 minutes

This segmentation reflects common retail visit missions and allows slopes to vary across behavioral regimes.

➢ **Segment-Level Regression Results**

Quick Trip

- Slope: 0.117
- R^2: 0.69

Short visits show a strong relationship between time and spending, but total spending remains limited. These customers typically enter with a predefined mission.

Regular Shopping

- Slope: 0.043
- R^2: 0.58

This segment represents the core customer base. Additional time translates into meaningful incremental purchases, making this group the most relevant from a revenue optimization perspective.

Long Stay

- Slope: 0.014
- R^2: 0.65

Long visits exhibit diminishing marginal returns. Additional time does not convert proportionally into spending, reflecting browsing behavior, decision fatigue, or non-purchasing time.

➢ **Overall Conclusions from Regression 2**

- Time spent in store is positively correlated with purchase value across all models.
- A single global regression masks important behavioral differences.
- Log transformation improves statistical validity but does not fully capture non-linearity.
- Time-based segmentation provides the most behaviorally accurate and business-relevant interpretation.

Regression 2 demonstrates that more time does not always mean proportionally more revenue, and that monetization efficiency depends strongly on visit type.